



数学建模国赛集训

优化模型概述

卢 鹏

成都理工大学 2026年7月11日

精勤求學 敦篤勵志 果毅力行 忠恕任事



内容提要

CONTENT

1

优化问题概述

2

建立优化模型的要点与技巧

3

常见优化模型

4

优化问题的经典算法

5

启发式算法与优化 (7.16)

6

计算机模拟与优化



1. 优化问题概述（从生活感受说起）

（1）先问大家一个问题：你们每天早上从宿舍出发，要在8点前赶到教学楼，通常会怎么选路线？

- 有人说走最近的路——这是在距离最短的目标下找最优解。
- 有人说骑车最快——这是在时间最少的目标下找最优解。
- 还有人说顺路买个早餐——这就变成了多目标优化：既要省时，又要吃饭。

（2）再举个例子。你们期末考试周，有5门课要复习，总共可用时间40小时。怎么分配时间，才能让总平均分最高？这也是优化问题——决策变量是每门课投入的时间，目标函数是总平均分，约束条件是总时间不超过40小时。

所以优化模型离我们一点都不远。只要一个问题涉及“选择”和“取舍”，它大概率就是个优化问题。



1. 优化问题概述

优化问题即在给定约束条件下，寻找使目标函数达到最优的决策变量取值。涵盖线性、整数、非线性、多目标、**图论**、**网络流**、**组合数学**等类型。这类问题在生产调度、交通控制、物流、管理科学和社会科学等有着广泛的用途。

备考重点提示

- 每年稳定至少 2 道核心优化大题，优化是国赛高频赛道；
- 趋势：传统线性规划+复杂非线性、多目标、时空动态优化逐年增多；
- 高频算法：线性规划、整数规划、遗传算法 GA、粒子群 PSO、差分进化 DE。



1. 优化问题概述（2023年中的优化问题）

1. A 题：定日镜场的优化设计（硬核连续空间优化）

- 优化目标：最大化镜场单位镜面年均输出热功率
- 优化变量：定日镜坐标、安装高度、镜场排布、镜尺寸、遮挡 / 阴影约束
- 模型类型：多约束非线性规划、几何光学耦合优化、蒙特卡洛效率计算
- 求解方法：遗传算法、粒子群 PSO、梯度寻优、区域网格遍历优化
- 约束：镜间遮挡、集热器截断、太阳角度时变、土地边界

2. C 题：蔬菜类商品自动定价与补货决策（多周期整数规划 + 动态优化）

- 优化目标：多日总利润最大化
- 优化变量：每日补货量、阶梯定价、损耗控制
- 模型类型：动态线性规划、库存优化、价格弹性优化
- 约束：货架容量、损耗率、需求与价格联动、多品类互斥补货



1. 优化问题概述（2024中的优化问题）

1. B 题：生产过程中的决策问题（多阶段生产调度优化）
 - 优化目标：最小化生产成本、最大化产能利用率
 - 模型：混合整数规划 MIP、工序调度、设备负荷均衡优化
 - 约束：工序先后、设备工时、原料库存、交付周期
2. C 题：农作物的种植策略（经典线性 / 整数规划优化）
 - 优化目标：7年总种植利润最大化
 - 变量：54块地块、多年份作物种植面积
 - 约束：轮作要求（每 3 年必种豆类）、地块类型适配、产量滞销损失、灌溉限制、田间管理复杂度
 - 拓展：不确定参数下鲁棒优化、多目标（收益 + 管理简便）优化



1. 优化问题概述（2025中的优化问题）

1. A 题：烟幕干扰弹的投放策略（多无人机协同投放优化）
 - 优化目标：遮蔽目标有效时长最大化、弹药消耗最小、多机协同覆盖最优
 - 变量：无人机起飞时刻、投放坐标、投放时序、多弹协同布设
 - 模型：多目标非线性优化、时空覆盖优化、约束规划
 - 求解：PSO、遗传算法、多起点全局寻优；强局部最优陷阱
 - 约束：无人机航程、飞行速度、弹扩散范围、目标移动轨迹、弹药总量限制



1. 优化问题概述（常用计算机软件）

一、Matlab/Mathematica/Maple/北太天元

数学软件，用于算法开发，数据分析、数值计算。特色**工具箱**。后者为首款国产科学计算软件（与MATLAB类似）。

二、Lingo/Gurobi（高效求解大规模优化问题）

求解优化问题的专门软件。编程简单。非线性可能求不出最优解。

三、Spss/SAS/SPSSPRO

数据统计分析软件，无须编程。后者区别于SPSS、SAS传统客户端模式的全新在线数据分析平台，全部算法均为国产自研。

四、Python

比较流行的程序设计语言。

五、c/c++、c#、Excel等

其他软件。



2. 建立优化模型的要点与技巧

参考优秀论文:2025-A196、2024-A092\A127\C038

一、准确理解题意

仔细读题，**反复读题**，弄清楚最终要解决的是什么问题？猜测出题人的想法！有**几个问**？是单目标还是**多目标**？拿不准就先试算或先做再**回头看**。

二、选择适当的决策变量

需要优化的目标是由哪些因素决定的？适当引入**决策变量**来表示这些因素。通常引入的决策变量有**0-1变量**，整数变量等。

三、设法用决策变量表示目标函数

在用决策变量表示目标函数时，有时需要引入**中间变量（辅助变量）**，有时要利用取整函数、符号函数、绝对值函数、分段函数等等。

四、仔细分析约束条件

决策变量满足的约束主要有两方面：一是自身应有的约束，如非负约束、取整约束等；二是题目要求及客观实际的约束。



2. 建立优化模型的要点与技巧

- 一、多目标问题的妥协：线性加权法、理想点法、极大极小法、优先等级法、序贯式算法等。
- 二、数据预处理：对输入数据进行清理、标准化和归一化处理，以提高模型的准确性和稳定性。
- 三、非线性规划的线性化：有时通过引入人工变量可将非线性规划问题线性化，提高求解的精度与计算的时间（需要有一定的经验和准备）。
- 四、初始点选择：对于一些优化算法，初始点的选择可能会影响求解的效率和结果。可以尝试不同的初始点，或者使用一些启发式方法来选择较好的初始点。
- 五、调整参数：许多优化算法都有一些参数需要调整，如步长、收敛准则等，通过试验和调整这些参数，可以获得更好的求解效果。
- 六、复杂规划问题的拆分：有时可将复杂的问题分成一系列较为简单的子问题求解。
- 七、验证模型：在求解完成后，对结果进行验证，确保其合理性和可行性。可以使用实际数据或模拟数据进行验证。
- 八、混合方法：有时候结合使用多种优化方法或技术可以获得更好的结果。例如，可以先使用启发式算法获得一个初始解，然后再使用精确算法进行进一步优化。
- 九、持续改进：根据实际应用中的反馈和新的需求，不断调整和改进模型。



2*. 非线性-线性化举例 (处理技巧)

1. 变量 $x \in [1, 2]$ 或 $x \in [10, 20]$ 怎么线性化表示?
2. 线性约束条件共有 n 个: $A_i X - \beta_i \leq 0, i = 1, 2, \dots, n$, 写出任取 k ($k \leq n$) 个进入模型的方法。

$$\begin{cases} 1 - U * (1 - y_1) \leq x, x \leq 2 + U * (1 - y_1) \\ 10 - U * (1 - y_2) \leq x, x \leq 20 + U * (1 - y_2) \\ y_1 + y_2 = 1 \\ y_1, y_2 = 0 \text{ or } 1 \end{cases}$$

$$\begin{cases} y_1 + y_2 + \dots + y_n = k \\ A_i X - \beta_i \leq U * (1 - y_i) \\ y_i = 0 \text{ or } 1, i = 1, 2, \dots, n \end{cases}$$

说明: U 是一个充分大的数 (比如1000) .

3. 两个0-1变量的乘积如何线性化表示? , 一般地, n 个0-1变量的乘积如何线性化表示?
4. 记 $y = b_1 b_2 x$ (b_1, b_2 为0-1变量, $x \in R$), 如何线性化?
5. 半连续变量 $x = 0$ 或 $100 \leq x \leq 200$ 如何线性表示? (自行思考3, 4, 5)



2*. 非线性-线性化举例 (处理技巧)

分段函数的一个一般处理方法，设有限区间上的有界分段函数

$$y = f(x) = \begin{cases} \phi_1(x), & x \in (a_1, b_1) \\ \phi_2(x), & x \in (a_2, b_2) \\ \dots\dots\dots \\ \phi_n(x), & x \in (a_n, b_n) \end{cases}$$

这里 $a_1 < b_1 \leq a_2 < b_2 \leq \dots \leq a_n < b_n$

则该分段函数与下面的模型等价：

$$\begin{cases} y = y_1 + y_2 + \dots + y_n \\ z_1 + z_2 + \dots + z_n = 1, z_i \in \{0, 1\} \\ a_i - x \leq U * (1 - z_i), x - b_i \leq U * (1 - z_i), \\ -U * z_i \leq y_i \leq U * z_i, \\ \phi_i(x) - U * (1 - z_i) \leq y_i \leq \phi_i(x) + U * (1 - z_i), \\ i = 1, 2, \dots, n \end{cases}$$

其他情况说明：

- (1) 若 $a_i = b_i$ 则为离散情况，以上模型仍然成立；
- (2) 以上结果可以推广到多元分段函数的情况。

- 说明：
- (1) z_i 为0-1变量，标识 y 取哪一段；
 - (2) y_i 的取值要么为0，要么为 ϕ_i ；
 - (3) U 是一个充分大的数（函数 y 的界）。



2*. 离散优化问题注意要点

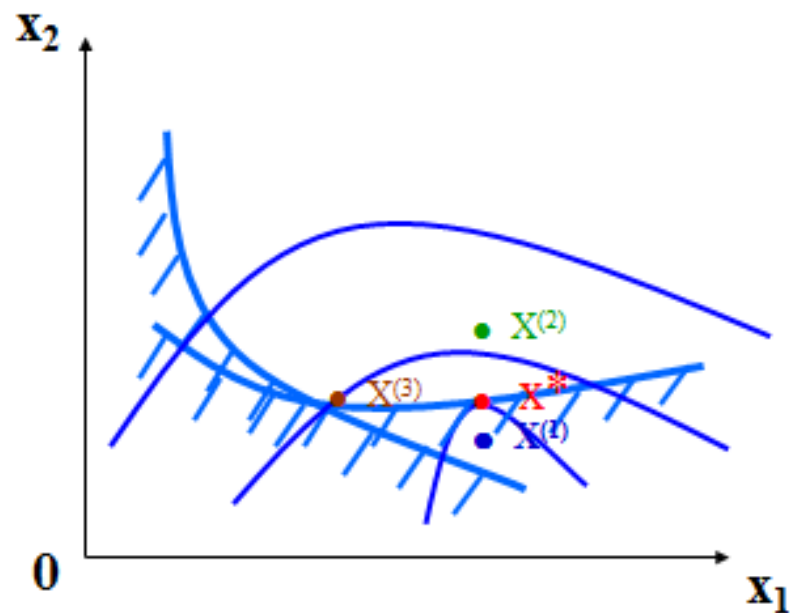
部分同学在求离散问题的最优解，喜欢的方法是先利用连续变量优化设计方法求连续变量的最优解，然后圆整到离散值上。此种方法的弊病就是可能得不到可行最优解，或所得的解不是离散最优解。

x^* 为连续变量最优解；

$x^{(1)}$ 是圆整后最近的离散点，但不可行；

$x^{(2)}$ 是最近的可行离散点，但不是离散最优点；

$x^{(3)}$ 是离散最优点。



离散变量优化难点：不存在指导搜索过程的最优性条件。

直接方法：枚举法（enumeration）可行。但可行点过多时，计算量大。

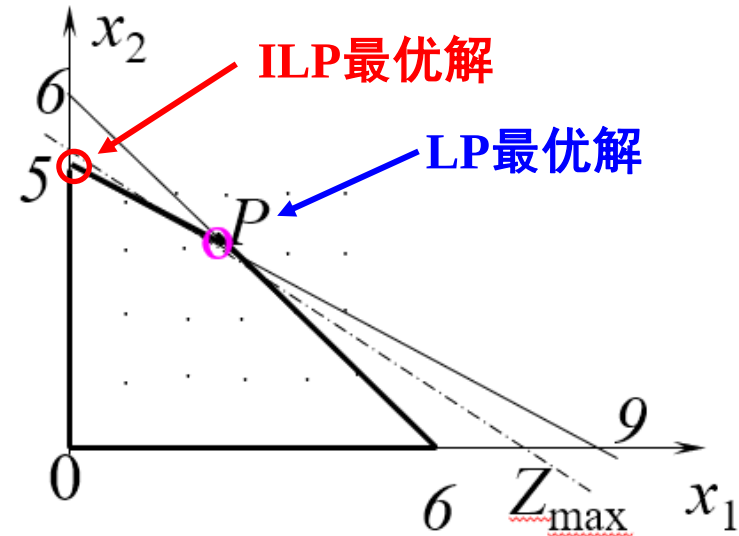


2*. 离散优化问题示例

ILP模型

$$\begin{aligned} \max \quad & z = 5x_1 + 8x_2 \\ \text{s.t.} \quad & x_1 + x_2 \leq 6 \\ & 5x_1 + 9x_2 \leq 45 \\ & x_1, x_2 \geq 0 \text{ 且为整数} \end{aligned}$$

去掉**整数限制**后，可行域为点 $(0,0)$, $(6,0)$, $(0,5)$, $P(2.25, 3.75)$ 围成的4边形



LP最优解P	P的舍入解	最靠近P的可行解	ILP最优解
$(2.25, 3.75)$	$(2, 4)$	$(2, 3)$	$(0, 5)$
$z=41.25$	不可行	$z=34$	$z=40$

结论： 从LP最优解经过简单的“移动”不一定得到ILP最优解。



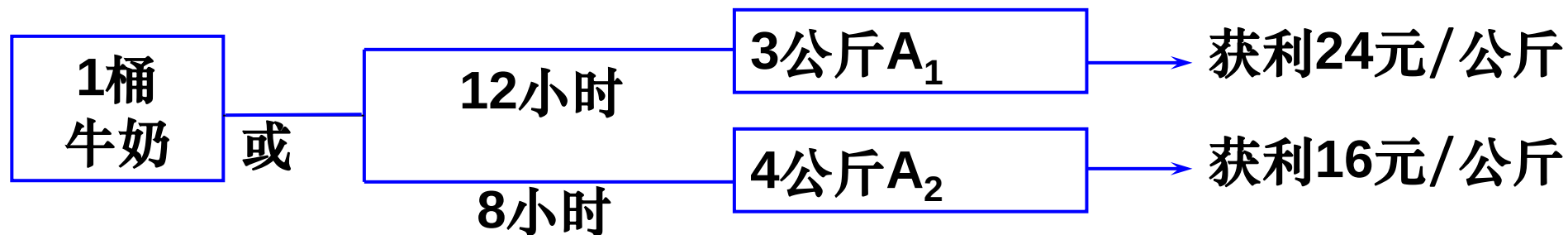
3. 常见优化模型

优化研究包含有限可行解的各类实际问题，在生活中应用广泛，核心应用领域是合理调配稀缺资源以提升生产效益。

涵盖线性规划、整数规划、0-1 规划、组合优化与图论等内容，数模竞赛高频题型包括下料、指派、选址、装箱、背包、最短路、TSP 旅行商及多目标规划等问题。



3.1 牛奶加工模型（讲一个LP经典案例）



每天

50桶牛奶

时间480小时

至多加工100公斤 A_1

制定生产计划，使每天获利最大？

- 35元可买到1桶牛奶，买吗？若买，每天最多买多少？
- 可聘用临时工人，付出的工资最多是每小时几元？
- A_1 的获利增加到 30元/公斤，应否改变生产计划？



3.1 牛奶加工模型（三要素）

引入决策变量

x_1 桶牛奶生产 A_1 , x_2 桶牛奶生产 A_2 （每天）

目标函数

生产 A_1 获利: $24 \times 3x_1$

生产 A_2 获利: $16 \times 4x_2$

每天获利总额: $z = 72x_1 + 64x_2$

约束条件

原料供应: $x_1 + x_2 \leq 50$

劳动时间: $12x_1 + 8x_2 \leq 480$

加工能力: $3x_1 \leq 100$

非负约束: $x_1, x_2 \geq 0$



3.2 下料问题模型

现要做100套钢架, 用长为2. 9m、 2. 1m和1. 5m的元钢各一根, 已知原料长7. 4m, 问如何下料, 使用的原材料最省?

分析: 最省的方式有两种

1. 总余料最少;

2. 总根数最少;

例举所有处理方式 →

原料截成所需 长度的根数		下料方法							
		①	②	③	④	⑤	⑥	⑦	⑧
所需根长	2.9m	2	1	1	1	0	0	0	0
	2.1m	0	2	1	0	3	2	1	0
	1.5m	1	0	1	3	0	2	3	4
剩余料头		0.1	0.3	0.9	0	1.1	0.2	0.8	1.4



3.2 下料问题模型

决策变量： x_i 表示按第 i 种办法下料的原材料根数，则问题的整数线性规划模型为：

$$\begin{aligned} \min f &= 0.1x_1 + 0.3x_2 + 0.9x_3 + 0x_4 + 1.1x_5 + 0.2x_6 + 0.8x_7 + 1.4x_8 \\ s.t. &\begin{cases} 2x_1 + x_2 + x_3 + x_4 \geq 100 \\ 2x_2 + 3x_3 + 3x_5 + 2x_6 + x_7 \geq 100 \\ x_1 + x_3 + 3x_4 + 2x_6 + 3x_7 + 4x_8 \geq 100 \\ x_j \in N, j = 1, 2, 3, 4, 5, 6, 7, 8 \end{cases} \end{aligned}$$

总余料最少

LINGO
直接求解

model:

Min=0.1*x1 + 0.3*x2 + 0.9*x3 + 0*x4 + 1.1* x5 + 0.2* x6 + 0.8*x7 + 1.4*x8;

2*x1 + x2 + x3 + x4 > 100;

2*x2 + 3*x3 + 3*x5 + 2*x6 + x7 > 100;

x1 + x3 + 3*x4 + 2*x6 + 3*x7 + 4*x8 > 100;

@gin(x1);@gin(x2);@gin(x3);@gin(x4);@gin(x5);@gin(x6);@gin(x7);@gin(x8);

end



3.3 指派问题模型

设有 n 件工作 $B_1, B_2, \dots, B_n$ 分派给 n 人 $A_1, A_2, \dots, A_n$ 去做, 每人只做一件工作且每件工作只派一个人去做, 设 A_i 完成 B_j 的工时为 c_{ij} , 问应如何分派才能完成全部工作的总工时最少?

解: 设 x_{ij} 为0-1变量, 其中1表示工作 B_j 分配给 A_i 去做, 0表示没有分配该项工作。

$$\min f = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}$$

$$\text{s.t.} \begin{cases} \sum_{i=1}^n x_{ij} = 1 & (j = 1, 2, \dots, n) \\ \sum_{j=1}^n x_{ij} = 1 & (i = 1, 2, \dots, n) \\ x_{ij} = 0 \text{ 或 } 1 & (i, j = 1, 2, \dots, n) \end{cases}$$

每件工作只派1人

每个人只做1件工作



3.3 指派问题模型

- 例如求6个人做6项工作的最优指派，收益情况如下表。

人	工作1	工作2	工作3	工作4	工作5	工作6
1	20	15	16	5	4	7
2	17	15	33	12	8	6
3	9	12	18	16	30	13
4	12	8	11	27	19	14
5	—	7	10	21	10	32
6	—	—	—	6	11	13

LINGO直接求解

```
model:
sets:
flight/1..6/;
assign(flight,flight):c,x;
endsets
max=@sum(assign:c*x);
@for(flight(i):@sum(flight(j):x(i,j))=1);
@for(flight(j):@sum(flight(i):x(i,j))=1);
@for(assign:@bin(x));
data:
c=20 15 16 5 4 7
   17 15 33 12 8 6
   9 12 18 16 30 13
  12 8 11 27 19 14
 -99 7 10 21 10 32
 -99 -99 -99 6 11 13;
enddata
end
```



3.4 选址问题模型

某公司拟定在武昌、汉口、汉阳建立专卖店，拟议中有7个地址：中商(A1)，亚贸(A2)，司门口(A3)，武广(A4)，步行街(A5)，二十一世纪(A6)，汉商(A7)，并规定：武昌至多2个，汉口汉阳至少1个，若选 A_i ，投资 b_i 元，每年可获利 c_i 元，总投资不超过 b 元，问如何选择地址使公司的年利润最大？

$$\text{解: } x_i = \begin{cases} 1, & \text{选择 } A_i \\ 0, & \text{否则} \end{cases}$$

$$\max f = \sum_{i=1}^7 c_i x_i$$

$$s.t. \begin{cases} \sum_{i=1}^7 b_i x_i \leq b \\ x_1 + x_2 + x_3 \leq 2 \\ x_4 + x_5 \geq 1 \\ x_6 + x_7 \geq 1 \\ x_i = 0 \text{ 或 } 1 \quad i = 1, 2, \dots, 7 \end{cases}$$



3.5 装箱问题模型

当你装一个箱子时，你会发现要使箱子尽可能装满不是一件很容易的事，你往往需要做些调整。从理论上讲，装箱问题是一个**很难的组合优化**问题，即使用计算机也是不容易解决的。

装箱问题在工业生产及日常生活中有广泛的用途，其应用在实际生活中无处不在，货物装运，服装裁剪，以及我们计算机科学中的存储分配、共享资源调度、文件存储都是装箱问题在实际应用中的体现。所以具有重要的研究价值。

问题描述：设有编号为1、...、 n 的 n 种物品，体积分别为 v_1 、 v_2 、...、 v_n 。将这 n 种物品装到容量都为 V 的若干箱子里（更一般的装箱问题还可以要求容量不是相同的）。约定这 n 种物品的体积均不超过 V ，即对于 $1 \leq i \leq n$ ，有 $0 < v_i \leq V$ 。不同的装箱方案所需要的箱子数目可能不同。成功装载是指能把所有物品都装入箱子。最优装载是指使用最少箱子的成功装载。



3.5 装箱问题模型

已知30个物品, 其中6个长0.51m, 6个长0.27m, 6个长0.26m, 余下12个长0.23m, 箱子长为1m, 问最少需多少个箱子才能把30个物品全部装进箱子。

解析: 装箱问题的数学表示如下(0-1规划模型)。

$$\min z(y) = \sum_{i=1}^n y_i$$

$$s.t. \begin{cases} \sum_{j=1}^n \omega_j x_{ij} \leq V y_i, i \in N = \{1, \dots, n\} \\ \sum_{i=1}^n x_{ij} = 1, j \in N \\ y_i = 0 \text{ 或 } 1, i \in N \\ x_{ij} = 0 \text{ 或 } 1, j \in N \end{cases}$$

此约束表示: 哪些物品装入箱子*i*

其中: w_j 代表第*j*个物品长度, V 代表箱子长度

表示每个物品有且仅能装入一个箱子

$y_i = 1$ 表示箱子*i*装入物品, 反之表示箱子*i*空着

$x_{ij} = 1$ 表示物品*j*装入箱子*i*, 反之表示物品*j*未放入箱子*i*



3.5 装箱问题模型

```
MODEL:
SETS:
    WP/w1..w30/:W;
    XZ/v1..v30/:Y;
    LINKS(WP,XZ):X;
ENDSETS
MIN=@SUM(XZ(I):Y(I));
C=1;!C是箱子长度;
@for(XZ:@bin(y)); !限制Y是0-1变量; @for(LINKS:@bin(X)); !限制X是0-1变量;
@for(WP(I):@sum(XZ(J):X(I,J))=1); !每个物品只能放入一个箱子;
@for(XZ(J):@SUM(WP(I):W(I)*X(I,J))<=C*Y(J)); !每个箱子内物品的总长度不超过箱子;
DATA: W=0.51,0.51,0.51,0.51,0.51,0.51,0.27,0.27,0.27,0.27,0.27,0.27,0.26,0.26,0.26,0.26,
        0.26,0.23,0.23,0.23,0.23,0.23,0.23,0.23,0.23,0.23,0.23,0.23,0.23,0.23;
ENDDATA
END
```

说明：本问题是属于的**一维装箱**问题。

更多的是带约束的装箱问题（分类）：如二维装箱问题（条形装箱问题、剪裁问题）、三维装箱问题、变容装箱问题、有色装箱问题、对偶装箱问题等。在竞赛中应注意区分和鉴别。



3.6 背包问题模型

一维0/1背包问题

有N件物品和一个容量为W的背包。第j件物品的价值是 c_j ，体积是 w_j 。求将哪些物品装入背包可在满足背包容量允许的前提下使价值总和最大？背包问题在经济管理、资源分配、投资决策、装载设计等领域有着重要的应用价值。

解：设为二进制变量，如果物品j被放入背包，则 $x_j=1$ ，否则为0。

背包问题的数学模型描述如下：

$$\begin{aligned} \max \quad & \sum_{j=1}^n c_j x_j \\ \text{s.t.} \quad & \begin{cases} \sum_{j=1}^n w_j x_j \leq W \\ x_j \in \{0, 1\}, j = 1, 2, \dots, n \end{cases} \end{aligned}$$



3.6 背包问题模型

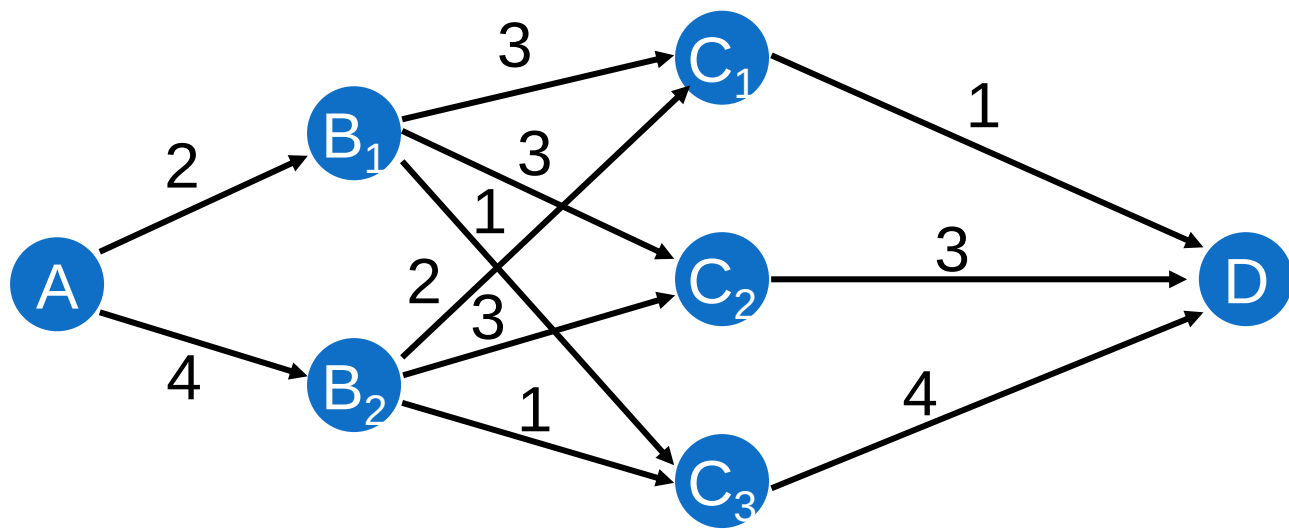
“超市大赢家”提供了50种商品作为奖品供中奖顾客选择, 车的容量为1000立方分米, 奖品 i 占用的空间为 w_i 立方分米, 价值为 c_i 元, 具体的数据可见程序, 如何选择总价值最大。

```
model:
sets:
    goods/w1..w50/:w,c,x;
endsets
data:
    c=220,208,198,192,180,180,165,162,160,158,155,130,125,122,120,118,115,110,
    105,101,100,100,98,96,95,90,88,82,80,77,75,73,72,70,69,66,65,63,60,58,56,50,30,20,15,10,8,5,3,1;
    w=80,82,85,70,72,70,66,50,55,25,50,55,40,48,50,32,22,60,30,32,40,38,35,32,25,
    28,30,22,50,30,45,30,60,50,20,65,20,25,30,10,20,25,15,10,10,10,4,4,2,1;
    capacity=1000;    !容量;
enddata
max=@sum(goods(j):c(j)*x(j));
@for(goods:@bin(x));    !限制x是0-1变量;
@sum(goods(J):w(j)*x(j))<capacity;
p=@sum(goods(J):x(j));    !所装物品总件数;
z=@sum(goods(J):w(j)*x(j));    !所装物品总体积;
end
```



3.7 最短路问题模型

设一个带权的图 $G=(V, E)$ 表示一交通运输网络，以图的各个顶点 V 代表一些城市，图的各条边 E 表示城市之间的交通运输路线，每条边的权值则表示此路线的长度或表示沿此路线运输所需花费的时间或运费等。运输路线往往是有方向性的，例如汽车运输有时会遇到单行路，而且不同的方向所花费的时间或代价可能不同，例如汽车的上山和下山、水路的顺水和逆水花费的时间或代价就不相同，所以交通运输网络一般是有向图，所谓最短路径问题指的是：如果从图中某顶点出发（此点称为源点），经图的边到达另一顶点（称之为目的点）的路径不止一条，如何找到一条路径使沿此路径上各边的权值之和为最小。



$$\begin{aligned} \min \quad & \sum_{(i,j) \in E} w_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{j:(i,j) \in E} x_{ij} - \sum_{j:(j,i) \in E} x_{ji} = \begin{cases} 1 & i=1 \\ -1 & i=n \\ 0 & i \neq 1, n \end{cases} \\ & x_{ij} \geq 0, (i, j) \in E \end{aligned}$$



3.7 最短路问题模型

现有A、B1、B2、C1、C2、C3、D共7个城市，点与点之间的连线表示城市间有道路相连。连线旁的数字表示道路的长度。求城市A到城市D的一条最短路。

```
model:
sets:
cities/A,B1,B2,C1,C2,C3,D/;
roads(cities,cities)/A B1,A B2,B1 C1,B1 C2,B1 C3,B2 C1,B2 C2,B2 C3,C1 D,C2 D,C3 D/:w,x;
endsets
data:
w=2 4 3 3 1 2 3 1 1 3 4;
enddata
n=@size(cities);    !城市的个数;
min=@sum(roads:w*x);
@for(cities(i)|i #ne#1 #and# i #ne#n:
@sum(roads(i,j):x(i,j))=@sum(roads(j,i):x(j,i)));
@sum(roads(i,j)|i #eq#1:x(i,j))=1;
@sum(roads(i,j)|j #eq#n:x(i,j))=1;
end
```

由于并非所有城市间都有道路直接连接，所以将弧具体列出

定义稀疏集合方法：枚举法



3.8 TSP问题模型

某人到四川旅游，先到成都市，然后从成都市出发租车自驾游到都江堰、青城山、黄龙、九寨沟、乐山大佛、峨眉山、海螺沟、西岭雪山、阆中古城、剑门蜀道、汶川这11个著名景点旅游观光，每个景点恰去一次，最后回到成都市，应如何安排旅游线路，使旅程最短？已知各景点之间的公路里程如表所示。

景区间的距离（单位：公里）



	成都	都江堰	青城山	黄龙	九寨沟	乐山大佛	峨眉山	海螺沟	西岭雪山	阆中古城	剑门蜀道	汶川
成都	0	65	67	384	456	154	213	287	120	262	273	135
都江堰	65	0	14	315	484	208	261	358	134	320	283	73
青城山	67	14	0	319	485	199	229	280	87	321	286	82
黄龙	384	315	319	0	129	515	530	617	438	465	320	239
九寨沟	456	484	485	129	0	619	630	784	560	395	298	552
乐山大佛	154	208	199	515	619	0	50	316	256	395	530	306
峨眉山	213	261	229	530	630	50	0	305	245	434	569	331
海螺沟	287	358	280	617	784	316	305	0	296	549	684	381
西岭雪山	120	134	87	438	560	256	245	296	0	385	362	204
阆中古城	262	320	321	465	395	395	434	549	385	0	133	391
剑门蜀道	273	283	286	320	298	530	569	684	362	133	0	320
汶川	135	73	82	239	552	306	331	381	204	391	320	0



3.8 TSP问题模型

以各景点（包括成都市）为图的顶点，两景点间的公路为图相应两顶点间的边，这样就得到一个完全图。问题就是从成都出发，每个景点恰去一次，最后回到成都市。通俗来讲就是：找从一点出发不重复地走过所有的结点，最后又回到原出发点的路径的问题，在图论中称为“**哈密尔顿回路问题**”。

清晰问题

“哈密尔顿回路问题”与“欧拉回路问题”的**不同点**

- ☆ “哈密尔顿回路问题”是访问每个**结点**一次，而“欧拉回路问题”是访问每条**边**一次。
- ☆ 对图G是否存在“欧拉回路”，欧拉已给出充分必要条件，而对图G是否存在“哈密尔顿回路”至今仍未找到满足该问题的充分必要条件。
- ☆ 欧拉回路问题也称为“中国邮递员问题”。中国数学家管梅谷于1960年首先研究并给出算法。



3.8 TSP问题模型

设城市的个数为 n (本题为12), d_{ij} 是两个城市 i 与 j 之间的距离。

$$\min Q = \sum_{i \neq j} d_{ij} x_{ij}$$

$$s.t. \begin{cases} \sum_{j=1}^n x_{ij} = 1, & i = 1, 2, \dots, n \end{cases}$$

表示对每个顶点, 出去的路只有一条

$$s.t. \begin{cases} \sum_{i=1}^n x_{ij} = 1, & j = 1, 2, \dots, n \end{cases}$$

表示对每个顶点, 进去的路只有一条

$$u_i - u_j + nx_{ij} \leq n - 1, u_i, u_j \geq 0, \quad i = 1, 2, \dots, n; j = 2, \dots, n; i \neq j$$

$$x_{ij} \in \{0, 1\}$$

除起点和终点外, 各边不构成圈

取1代表从城市 i 驾车到城市 j , 取0代表没走

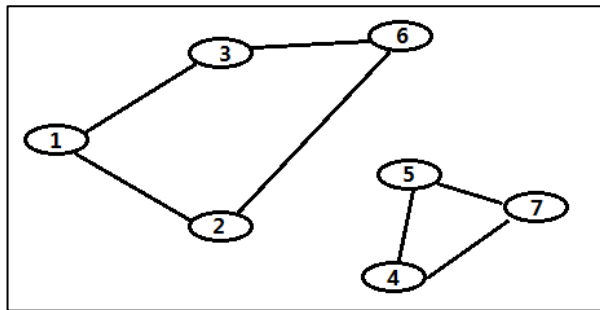
核心思想
破圈



3.8 TSP问题模型

解释： 假设如图所示对于子巡回4-5-7-4，由约束条件有

$$\begin{cases} u_4 - u_5 + nx_{45} \leq n-1 \\ u_5 - u_7 + nx_{57} \leq n-1 \\ u_7 - u_4 + nx_{74} \leq n-1 \\ x_{45} = x_{57} = x_{74} = 1 \end{cases}$$



公式相加，化简 $\longrightarrow n \leq n-1$

矛盾，故旅游路线中不存在子巡回。

同时： 对于含起点1的整体巡回1-3-6-2-1，
由于约束 $j \geq 2$ 不包含起点1，故不会产生矛盾。

MODEL:

SETS:

city/1..12/:u;

link(city,city):d,x;

ENDSETS

DATA:

d=0	65	67	384	456	154	213	287	120	262	273	135
65	0	14	315	484	208	261	358	134	320	283	73
67	14	0	319	485	199	229	280	87	321	286	82
384	315	319	0	129	515	530	617	438	465	320	239
456	484	485	129	0	619	630	784	560	395	298	552
154	208	199	515	619	0	50	316	256	395	530	306
213	261	229	530	630	50	0	305	245	434	569	331
287	358	280	617	784	316	305	0	296	549	684	381
120	134	87	438	560	256	245	296	0	385	362	204
262	320	321	465	395	395	434	549	385	0	133	391
273	283	286	320	298	530	569	684	362	133	0	320
135	73	82	239	552	306	331	381	204	391	320	0

ENDDATA

N=@SIZE(city);

MIN=@SUM(link:d*x);

@FOR(city(k):@SUM(city(i)|i#NE#k:x(i,k))=1);

@FOR(city(k):@SUM(city(j)|j#NE#k:x(k,j))=1);

@FOR(link:@BIN(x));

@FOR(link(i,j)|i#NE#j#and#i#GT#1:u(i)-u(j)+N*x(i,j)<N-1);

END

LINGO程序



4. 优化问题的经典算法

优化问题的难点往往在于它的算法设计，许多优化问题的计算量是非常大的，甚至是指数型的。诚然，我们已经有了大量的经典的算法，在解决离散问题时，我们经常使用分支定界法，割平面法，动态规划法等一些有效的办法，有时我们也会运用一些随机搜索的办法，如模拟退火法，遗传算法等。应该知道，这些通用的方法，都会有它的局限性，有的时候表现并不好。作为建模比赛，我们必须通过研究具体问题中的特殊关系，提出适合特殊一类问题的好的算法，即要思考**计算精度**和**计算效率**两个方面。

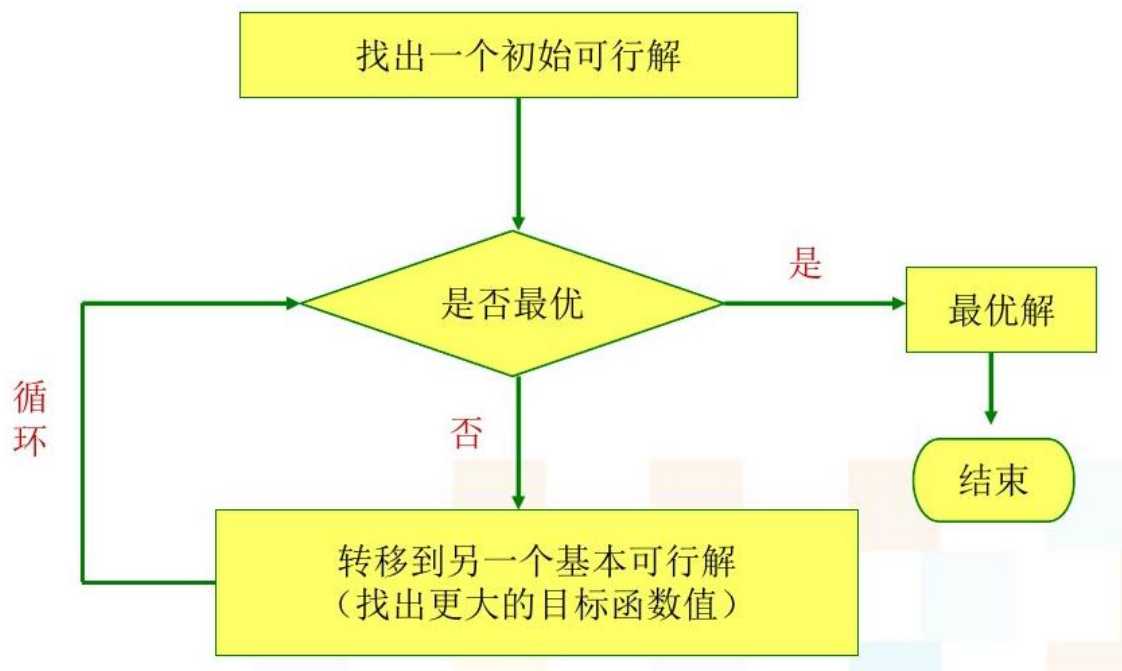
本内容接下来会介绍一些经典的最优化问题的求解方式：单纯形法（思想）、回溯法、分支定界法、贪婪算法、动态规划等。



4.1 单纯形法

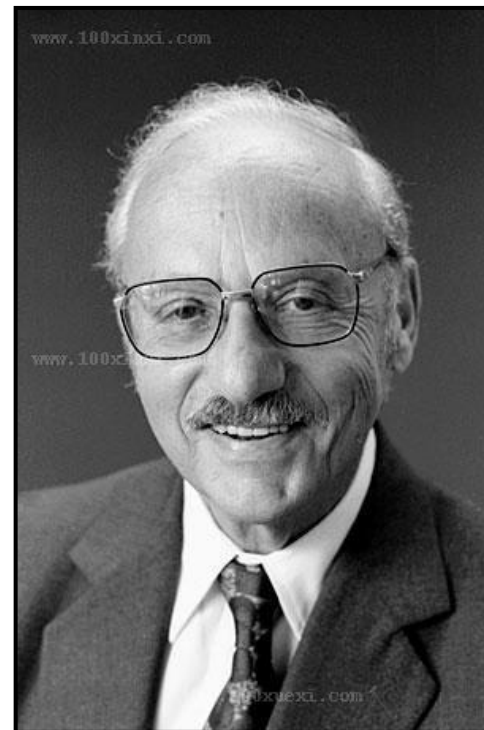
注释： 图形法求解线性规划只适用于二维或三维的情况，但高维情况不在适用。

方法： 单纯形法，1947年美国数学家George Bernard Dantzig提出。



核心思想
顶点对应
变量替换
单调不减

运筹学
最优化理论



1914-2005 线性规划之父



4.2 回溯法

许多离散问题的求解，需要搜索满足一定约束条件的所有可能的情况，找出合适的目标。回溯法有“**通用解题法**”之美称，是一种比枚举“聪明”的效率更高的搜索技术。

回溯是一种试探求解的方法：通过对问题的归纳分析，找出求解问题的一个线索，沿着这一线索往前试探，若试探成功，即得到解；若试探失败，就逐步往回退，换其它路线再往前试探。

回溯法可以形象地概括为“向前走，碰壁回头”，若往前走不可能得到解，就回溯，退一步另找线路，这样可省去大量的无效操作，提高搜索效率。

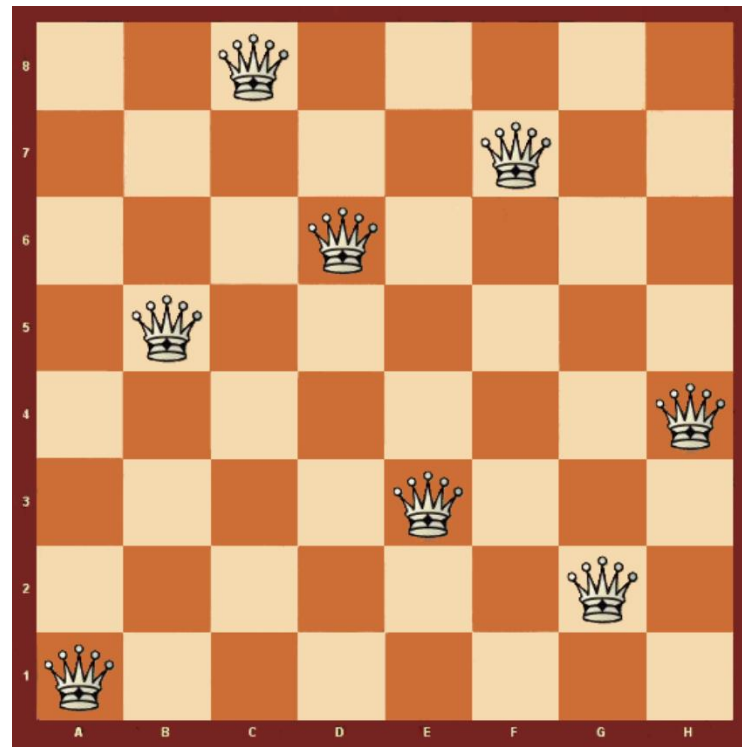


4.2 回溯法的经典案例

八皇后问题（英文：Eight queens），是由国际西洋棋棋手马克斯·贝瑟尔于1848年提出的问题，是回溯算法的典型案例。

问题表述为：在 8×8 格的国际象棋上摆放8个皇后，使其不能互相攻击，即任意两个皇后都不能处于同一行、同一列或同一斜线上，问有多少种摆法。

高斯认为有76种方案。1854年在柏林的象棋杂志上不同的作者发表了40种不同的解，后来有人用图论的方法解出92种结果。如果经过 ± 90 度、 ± 180 度旋转，和对角线对称变换的摆法看成一类，共有42类。

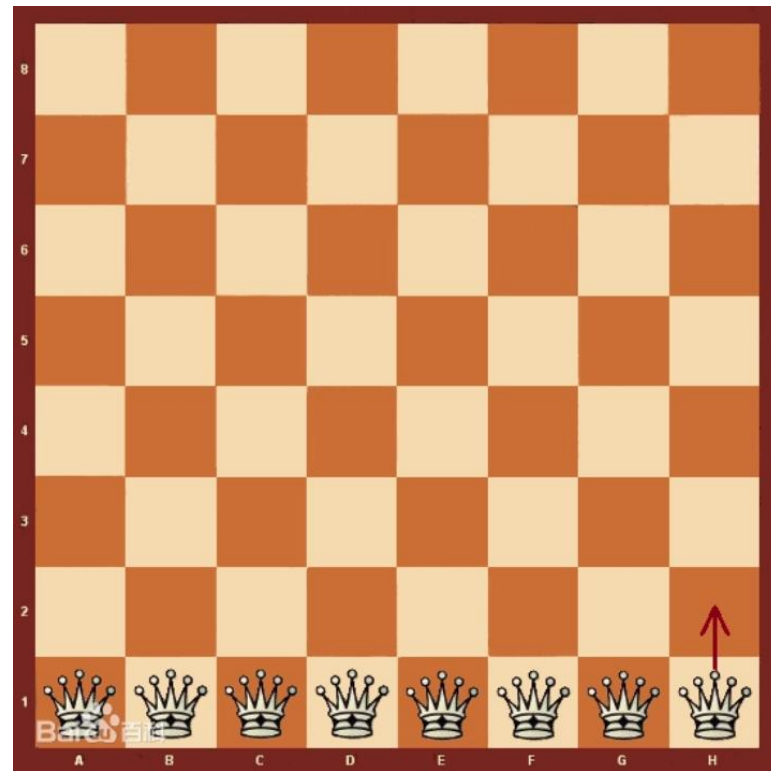




4.2 回溯法的经典案例

穷举法思路： 八皇后问题如果用穷举法需要尝试 $8^8=16777216$ 种情况。每一列放一个皇后，可以放在第1行，第2行，……，直到第8行。

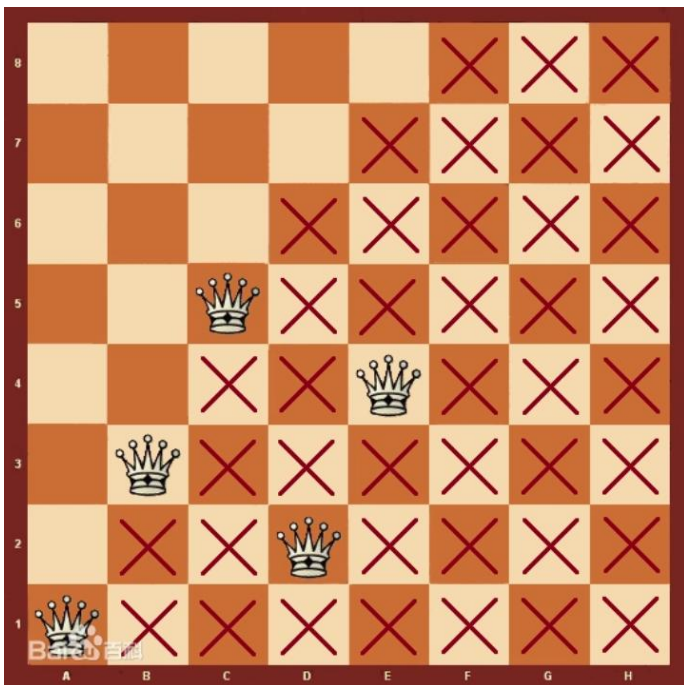
穷举的时候从所有皇后都放在第1行的方案开始，检验皇后之间是否会相互攻击。如果会，把列H的皇后挪一格，验证下一个方案。移到底了就“进位”到列G的皇后挪一格，**列H的皇后重新试过全部的8行**。这种方法是非常低效，因为它并不是哪里有冲突就调整哪里，而是盲目地按既定顺序枚举所有的可能方案。





4.2 回溯法的经典案例

回溯算法思路：回溯算法优于穷举法。将列A的皇后放在第一行以后，列B的皇后放在第一行已经发生冲突。这时候不必继续放列C的皇后，而是调整列B的皇后到第二行，继续冲突放第三行，不冲突了才开始进入列C。如此可依次放下列A至E的皇后，如左图所示。将每个皇后往右边横向、斜向攻击的点位用叉标记，发现列F的皇后无处安身。这时**回溯到列E的皇后**，将其位置由第4行调整为第8行，进入列F，发现皇后依然无处安身，**再次回溯列E**。此时列E已经枚举完所有情况，**回溯至列D**，将其由第2行移至第7行，再进入列E继续。按此算法流程最终找到如右图所示的解，成功在棋盘里放下了8个“和平共处”的皇后。继续找完全部的解共92个。



回溯算法求解八皇后问题的**原则**是：有冲突解决冲突，没有冲突往前走，无路可走往回退，走到最后是答案。为了加快有无冲突的判断速度，可以给每行和两个方向的每条对角线是否有皇后占据建立标志数组。放下一个新皇后做标志，回溯时挪动一个旧皇后清除标志。



4.3 分支定界法

引入概念：**松弛问题**。

以整型优化问题为例：

$$\begin{aligned} \max Z &= x_1 + 5x_2 \\ \text{s.t.} \quad &\begin{cases} x_1 - x_2 \geq -2 \\ 5x_1 + 6x_2 \leq 30 \\ x_1 \leq 4 \\ x_1, x_2 \geq 0 \text{ 且全为整数} \end{cases} \end{aligned}$$

松弛问题

$$\begin{aligned} \max Z &= x_1 + 5x_2 \\ \text{s.t.} \quad &\begin{cases} x_1 - x_2 \geq -2 \\ 5x_1 + 6x_2 \leq 30 \\ x_1 \leq 4 \\ x_1, x_2 \geq 0 \end{cases} \end{aligned} \quad \underline{z} \leq z^* \leq \bar{z}$$

分支定界法基本思想：设有最大化的整型优化**问题A**，相应**有松弛问题B**，从解松弛问题B开始，若其最优解**不符合A的整数条件**，那么B的最优目标函数必是A的最优目标函数 z^* 的**上界**，记作 $\bar{z}$ ；而A的任意可行解的目标函数值将是 z^* 的**一个下界**，记作 $\underline{z}$ 。

分支定界法就是将B的可行域分成子区域(称为分枝)的方法，逐步减小 $\bar{z}$ ，增大 $\underline{z}$ ，最终求到 z^* 。



4.3 分支定界法

三个基本操作：

(1) **分枝**: 在松弛问题B的最优解中任选一个不符合整数条件的变量 x_j , 其值为 b_j , 以 $[b_j]$ 表示小于 b_j 的最大整数。构造两个约束条件

$$x_j \leq [b_j] \quad \text{and} \quad x_j \geq [b_j] + 1$$

将这两个约束条件, 分别加入问题B, 求两个后继规划问题B1和B2, 不考虑整数条件求解这两个后继问题.

(2) **定界**: 以每个后继问题为一分枝标明求解结果, 在解的结果中, 找出最优目标函数值最大者作为新的上界. 从已符合整数条件的各分支中, 找出目标函数值为最大者作为新的下界, 若无, 则下界为0.

(3) **比较与剪枝**: 各分支的最优目标函数中若有小于 $\underline{Z}$, 则剪掉这枝; 若大于 $\underline{Z}$ 且不符合整数条件, 则重复前两步, 直到找到最优解。



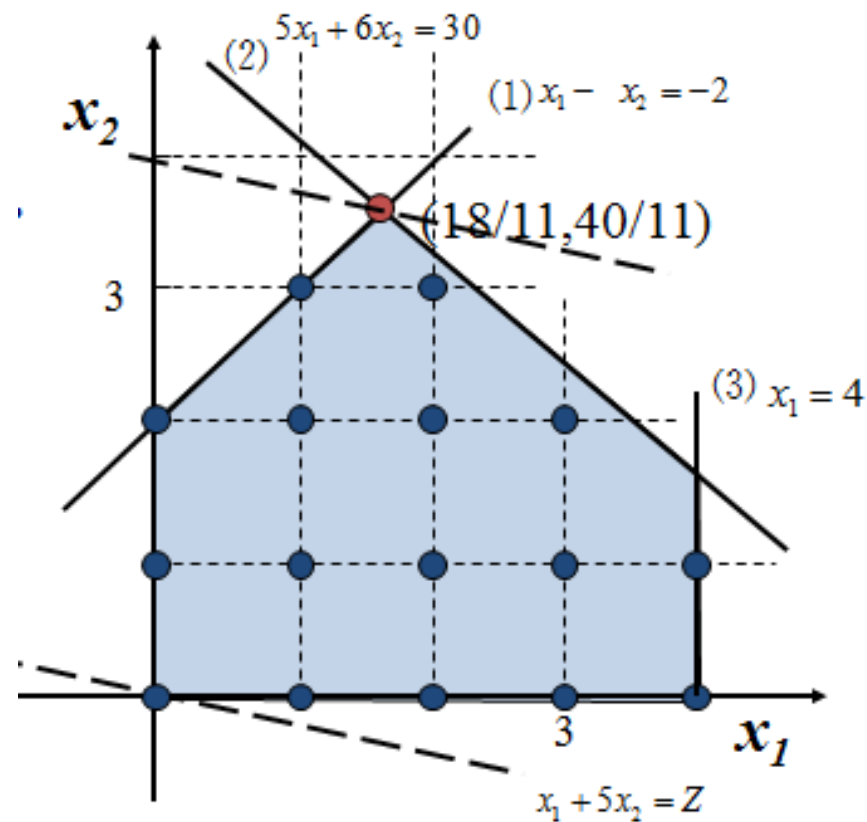
4.3 分支定界法案例

求出松弛问题最优解：

$$\begin{aligned} \max Z &= x_1 + 5x_2 \\ \text{s.t.} \quad &\begin{cases} x_1 - x_2 \geq -2 \\ 5x_1 + 6x_2 \leq 30 \\ x_1 \leq 4 \\ x_1, x_2 \geq 0 \end{cases} \end{aligned}$$

$$x^* = (x_1, x_2) = (18/11, 40/11)$$

$$Z^{(0)} = 218/11 \approx (19.8)$$



说明： $Z(0)$ 也是离散问题目标函数的上界。



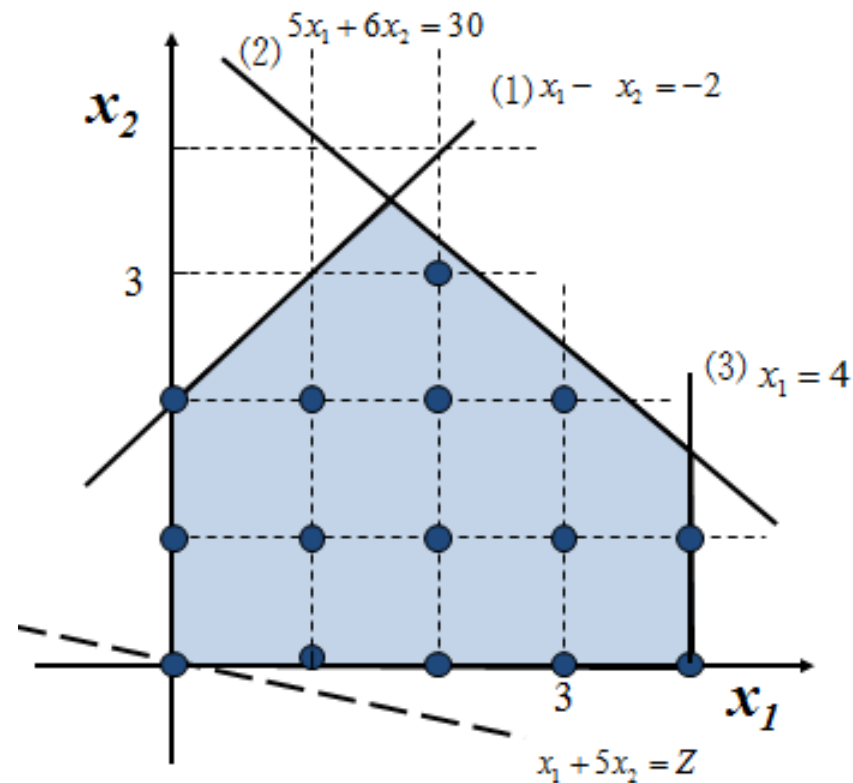
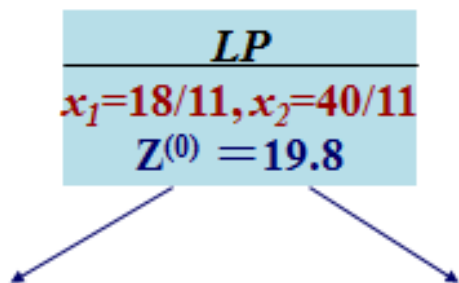
4.3 分支定界法案例

松弛问题最优解：

$$x^* = (x_1, x_2) = (18/11, 40/11)$$

$$Z^{(0)} = 218/11 \approx (19.8)$$

对于 $x_1 = 18/11 \approx 1.64$,
取值 $x_1 \leq 1, x_1 \geq 2$
对于 $x_2 = 40/11 \approx 3.64$,
取值 $x_2 \leq 3, x_2 \geq 4$



先将 (LP) 划分为 (LP1) 和 (LP2), 取 $x_1 \leq 1, x_1 \geq 2$ 。

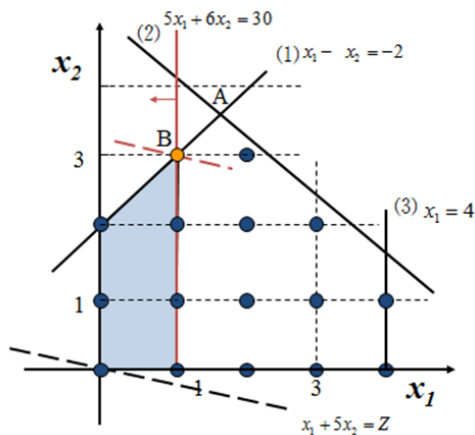


4.3 分支定界法案例

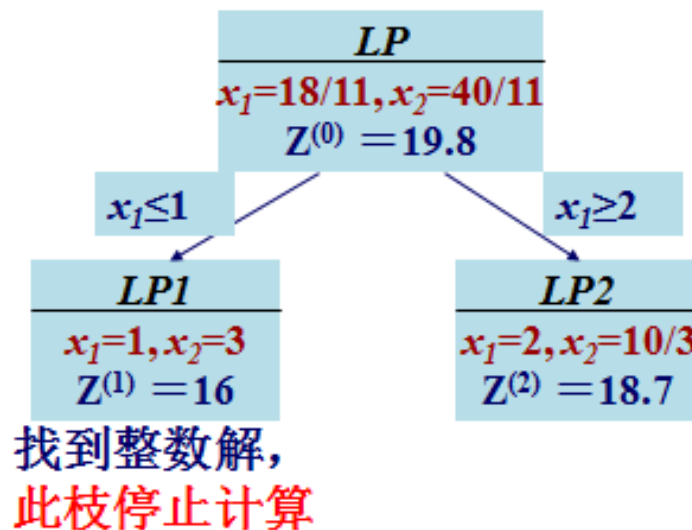
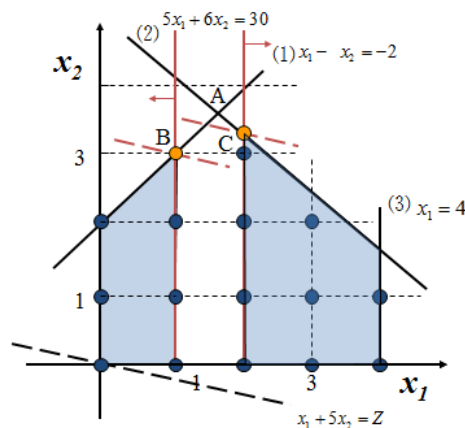
因为 $Z^{(2)} > Z^{(1)} = 16$, 原问题可能有比(16)更大的最优解;
但 x_2 不是整数, 故利用 $x_2 \leq 3$, $x_2 \geq 4$ 加入条件。

将 (LP) 划分为 (LP1) 和 (LP2), 取 $x_1 \leq 1$, $x_1 \geq 2$ 。

$$\begin{aligned} \max Z &= x_1 + 5x_2 \\ (LP1) \quad &\begin{cases} x_1 - x_2 \geq -2 \\ 5x_1 + 6x_2 \leq 30 \\ x_1 \leq 4 \\ x_1 \leq 1 \end{cases} \end{aligned}$$



$$\begin{aligned} \max Z &= x_1 + 5x_2 \\ (LP2) \quad &\begin{cases} x_1 - x_2 \geq -2 \\ 5x_1 + 6x_2 \leq 30 \\ x_1 \leq 4 \\ x_1 \geq 2 \end{cases} \end{aligned}$$



LP1在B点取得最优解。

$$x_1 = 1, x_2 = 3, Z^{(1)} = 16$$

LP2在C点取得最优解。

$$x_1 = 2, x_2 = 10/3, Z^{(2)} = 56/3 \approx 18.7$$

现在只要求出 (LP1) 和 (LP2) 的最优解即可。



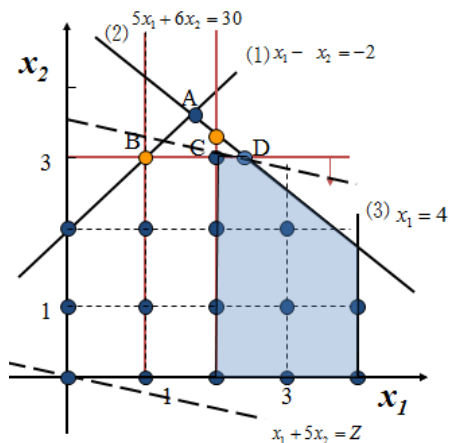
4.3 分支定界法案例

因为 $Z^{(21)} > Z^{(1)}$ ， $x_1 = 2.4$ 不是整数，可继续分枝，令 $x_1 \leq 2$ ， $x_1 \geq 3$ 。

将 (LP2) 划分为 (LP21) 和 (LP22)，取 $x_2 \leq 3, x_2 \geq 4$ 。 LP21在D点取得最优解。

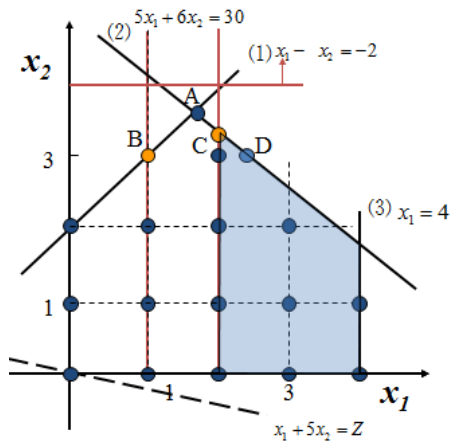
$$\max Z = x_1 + 5x_2$$

$$(LP21) \begin{cases} x_1 - x_2 \geq -2 \\ 5x_1 + 6x_2 \leq 30 \\ x_1 \leq 4 \\ x_1 \geq 2 \\ x_2 \leq 3 \end{cases}$$



$$\max Z = x_1 + 5x_2$$

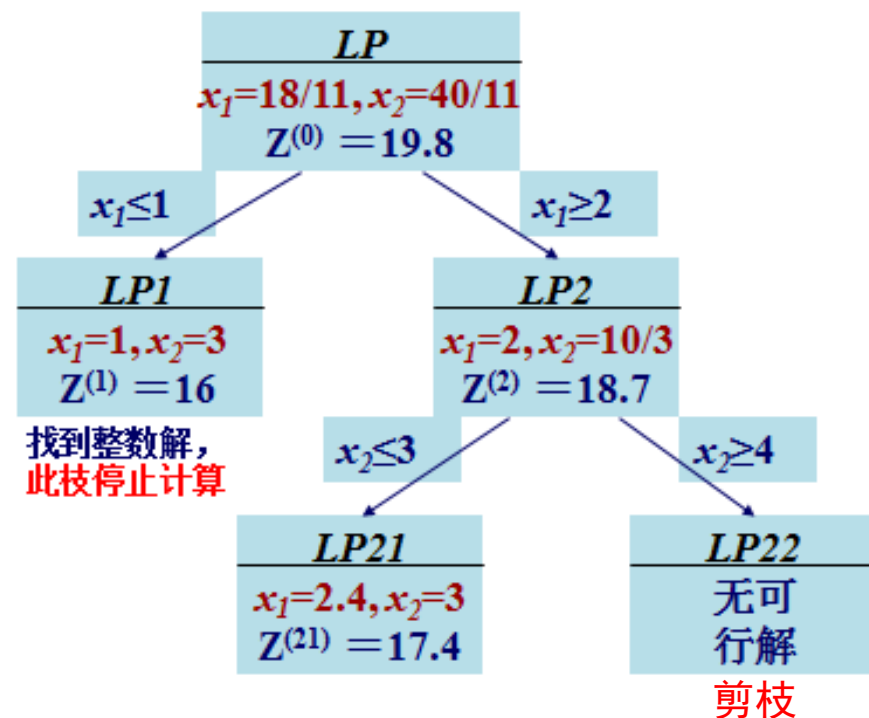
$$(LP22) \begin{cases} x_1 - x_2 \geq -2 \\ 5x_1 + 6x_2 \leq 30 \\ x_1 \leq 4 \\ x_1 \geq 2 \\ x_2 \geq 4 \end{cases}$$



$$x_1 = 12/5 = 2.4, x_2 = 3,$$

$$Z^{(21)} = 87/5 = 17.4$$

LP22, 无可行解, 不再分枝。



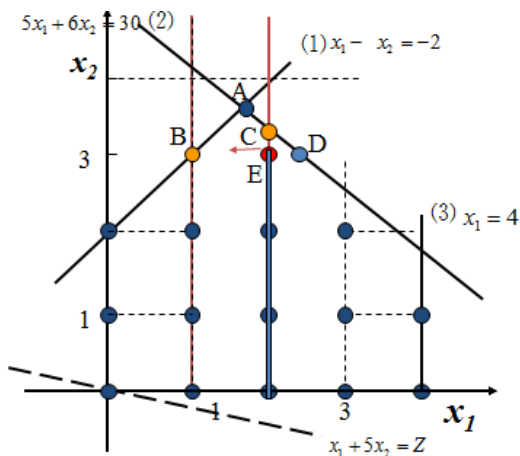


4.3 分支定界法案例

将 (LP21) 划分为 (LP211) 和 (LP222), 取 $x_1 \leq 2, x_1 \geq 3$ 。

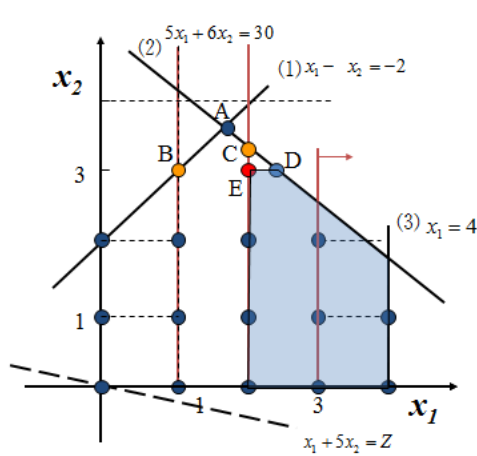
$$\max Z = x_1 + 5x_2$$

$$(LP211) \begin{cases} x_1 - x_2 \geq -2 \\ 5x_1 + 6x_2 \leq 30 \\ x_1 \leq 4 \\ x_1 \geq 2 \\ x_2 \leq 3 \\ x_1 \leq 2 \end{cases}$$



$$\max Z = x_1 + 5x_2$$

$$(LP212) \begin{cases} x_1 - x_2 \geq -2 \\ 5x_1 + 6x_2 \leq 30 \\ x_1 \leq 4 \\ x_1 \geq 2 \\ x_2 \leq 3 \\ x_1 \geq 3 \end{cases}$$



如图所示, LP211在E点取得最优解:

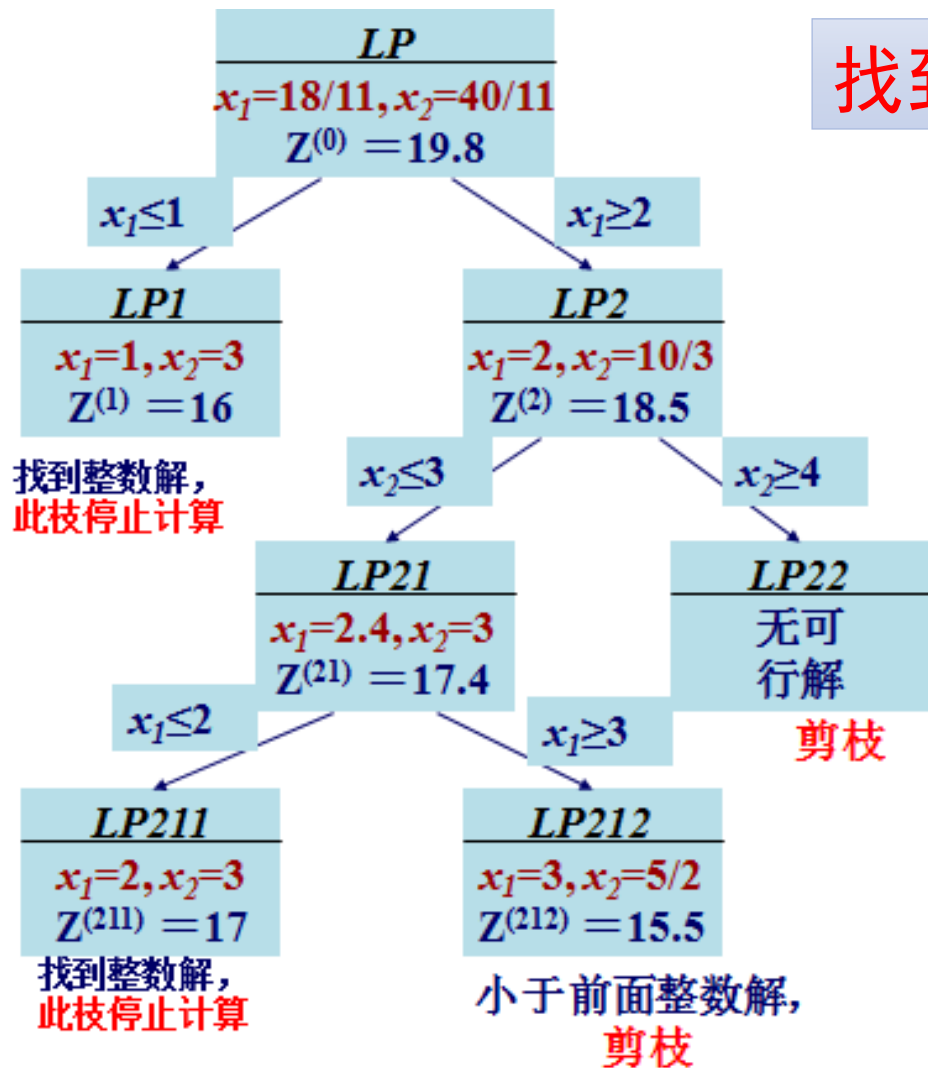
$$x_1 = 2, x_2 = 3, \\ Z^{(211)} = 17$$

如图所示, LP211在F点取得最优解:

$$x_1 = 3, x_2 = 2.5, \\ Z^{(212)} = 31/2 = 15.5$$



4.3 分支定界法案例



找到最优解

几点注意事项：

- (1) 若分枝后得到整数解，则这枝不必再分枝；
- (2) 若分枝后得到非整数解，如果比整数解更好，则这枝继续分枝；
- (3) 若分枝后得到非整数解，如果比整数解更差，则这枝不必再分枝。



4.4 贪婪算法

贪婪法是一种**不追求最优解**，只希望得到较为满意解的方法。贪婪法一般可以快速得到满意的解，因为它省去了为找最优解要穷尽所有可能而必须耗费的大量时间。贪婪法常以当前情况为基础作最优选择，而不考虑各种可能的整体情况。在贪婪算法（greedy method）中采用逐步构造最优解的方法。在每个阶段，都作出一个看上去最优的决策（**在一定的标准下**）。决策一旦作出，就不可再更改。作出贪婪决策的依据称为贪婪准则（greedy criterion）。

例如平时购物找钱时，为使找回的零钱的硬币数最少，不考虑找零钱的所有各种发表方案，而是从最大面值的币种开始，按递减的顺序考虑各币种，先尽量用大面值的币种，当不足大面值币种的金额时才去考虑下一种较小面值的币种。这就是在使用**贪婪法**。这种方法在这里总是最优，是因为银行对其发行的硬币种类和硬币面值的巧妙安排。

但如果只有面值分别为1、5和11单位的硬币，而希望找回总额为15单位的硬币。按贪婪算法，应找1个11单位面值的硬币和4个1单位面值的硬币，共找回5个硬币。但最优的解应是3个5单位面值的硬币。



4.4 贪婪算法

NF (Next Fit 下次适应) 算法: 按照物体给定的顺序装箱, 把物品 w_i 放到它第一个能放进去的箱子中。 B_j 是具有最大下标的使用过的箱子, 若 w_i 的长度不大于 B_j 的剩余长度, 则把 w_i 放入 B_j , 否则把 w_i 放入一个新的箱子 B_{j+1} , 且 B_j 在以后的装箱中不再使用。

FF (First Fit 首次适应) 算法: 按照物体给定的顺序装箱: 把物品 w_i 放到第一个箱子中。 $B_1 B_2 \cdots B_j$ 是当前已经使用过的箱子, 在这些箱子中找一个长度不小于 w_i 且下标最小的箱子, 将放入 w_i , 如果不存在这样的箱子, 则另开一个新箱子 B_{j+1} , 将 w_i 放入 B_{j+1} 中。

FFD (降序首次适应) 算法: 先将物体按长度从大到小排序, 然后按FF算法对物体装箱. 不失一般性, 对 n 件物品的体积按从大到小排好序, 即有 $v_1 \geq v_2 \geq \cdots \geq v_n$, 然后按排序结果对物品重新编号即可。



4.4 贪婪算法案例

利用FFD算法完成下题：

设有6种物品，它们的体积分别为：60、45、35、20、20和20单位体积，箱子的容积为100个单位体积。

解析过程：

步骤一：排序 $60 > 45 > 35 > 20 = 20 = 20$ ，按顺序编号①、②、③、④、⑤、⑥；

步骤二：将①放入第一个箱子中；

步骤三：②不能放入第一个箱子，将②放入第二个箱子中；

步骤四：将③放入第一个箱子中；

步骤五：④不能放入第一个箱子，将④放入第二个箱子中；

步骤六：⑤不能放入第一个箱子，将⑤放入第二个箱子中；

步骤七：⑥不能放入第一、二个箱子，将⑥放入第三个箱子中；

说明：第一个箱子装入 $60+35$ ，第二个箱子装入 $45+20+20$ ，第三个箱子装入20. (非最优解)

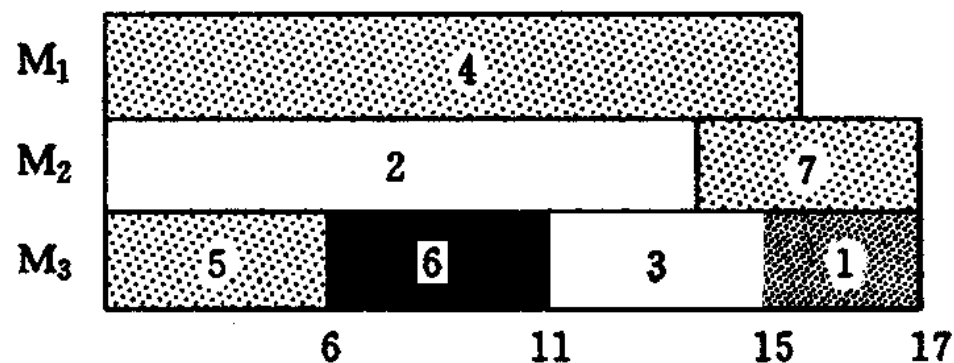


4.4 贪婪算法案例

多处理器调度问题：设有 n 个独立的作业 $\{1, 2, \dots, n\}$ ，由 m 台相同的机器进行加工处理。作业 i 所需的处理时间为 t_i 。现约定，任何作业可以在任何一台机器上加工处理，但未完工前不允许中断处理。任何作业不能拆分成更小的子作业。多机调度问题要求给出一种作业调度方案，使所给的 n 个作业在尽可能短的时间内由 m 台机器加工处理完成。

[分析]这个问题可以看成装箱问题(NP完全问题)。对于这一类问题，用贪婪选择策略有时可以设计出较好的近似算法。采用**最长处理时间作业优先**的贪婪选择策略可以设计出解多处理器调度问题的较好的近似算法。按此策略，当 $n \leq m$ 时，我们只要将机器 i 的 $[0, t_i]$ 时间区间分配给作业 i 即可。当 $n > m$ 时，我们首先将 n 个作业依其所需的处理时间**从大到小排序**。然后**依此顺序将作业分配给空闲的处理器**。

例如，设7个独立作业 $\{1, 2, 3, 4, 5, 6, 7\}$ 由3台机器 M_1, M_2 和 M_3 来加工处理。各作业所需的处理时间分别为 $\{2, 14, 4, 16, 6, 5, 3\}$ 。按贪婪算法产生的作业调度如图所示，所需的加工时间为17。





4.5 迪杰斯特拉 (Dijkstra)算法

Dijkstra于1959年提出了解决最短路问题的一般算法，此算法可按边的权值由小到大的次序，通过**贪婪选择**，逐步得到由给定源点到图的其余各点间的最短路径。其**基本作法**是，设置一个顶点集合 S ，一个顶点属于集合 S 当且仅当从源到该顶点的最短路径长度已知。初始时， S 中仅含有源。设 v_i 是 G 的某一个顶点，我们把从源到 v_i 且中间只经过 S 中顶点的路称为从源到 v_i 的路径，并用数组 $dist$ 来记录当前 S 中每个顶点所对应的最短路径长度。

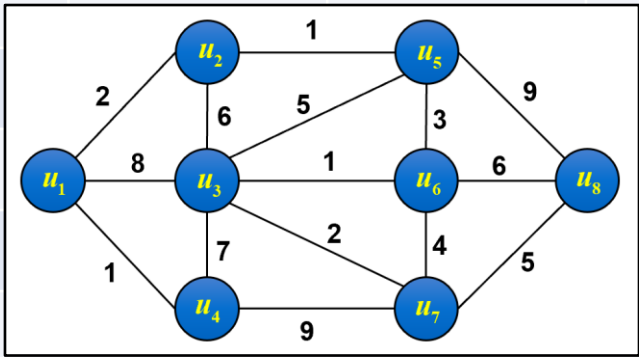
Dijkstra算法每次从 $V-S$ 中取出具有最短路径长度的顶点 v_i ，将 v_i 添加到 S 中，同时检查 v_i 添加到 S 中后，源点到 $V-S$ 中各顶点的距离是否可以通过源点到 v_i 的路径而缩短，如果缩短则对数组 $dist$ 作必要的修改。一旦 S 包含了所有 V 中顶点， $dist$ 就记录了从源到所有其他顶点之间的最短路径长度。



4.5.1 迪杰斯特拉算法案例

Di jkstra算法的具体步骤：

迭代次数	u1	u2	u3	u4	u5	u6	u7	u8
1	(0)	∞	∞	∞	∞	∞	∞	∞
2		2	8	(1)	∞	∞	∞	∞
3		(2)	8		∞	∞	10	∞
4			8		(3)	∞	10	∞
5			8			(6)	10	12
6			(7)				10	12
7							(9)	12
8								(12)
最短距离	0	2	7	1	3	6	9	12
前驱顶点	u1	u1	u6	u1	u2	u5	u3	u5

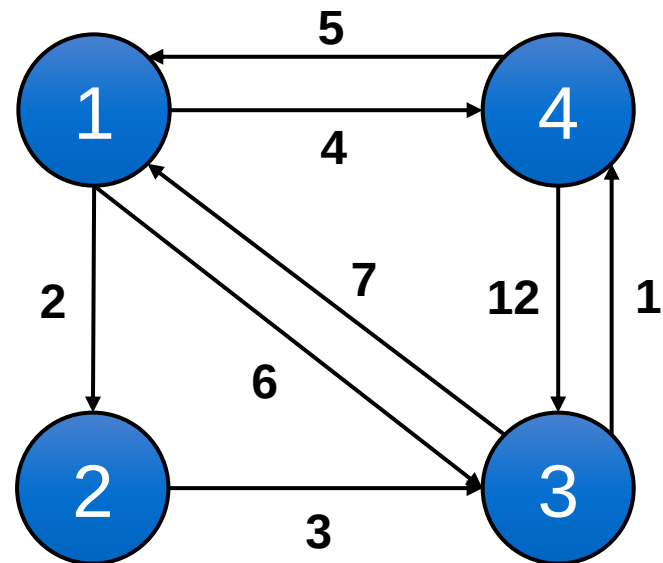


说明：带括号的是永久标号，括号内的数字就是从顶点u1到各相应顶点的最短路程。



4.6 弗洛伊德算法案例

暑假，小西准备去一些城市旅游。有些城市之间有公路，有些城市之间则没有，如图所示。为了节省经费以及方便计划旅程，小西希望在出发之前知道任意两个城市之间的最短路程。这个问题这也被称为“多源最短路径”问题。



	1	2	3	4
1	0	2	6	4
2	∞	0	3	∞
3	7	∞	0	1
4	5	∞	12	0

初始路程
情况

	1	2	3	4
1	0	2	6	4
2	∞	0	3	∞
3	7	9	0	1
4	5	7	11	0

只允许经过
1号顶点

	1	2	3	4
1	0	2	5	4
2	∞	0	3	∞
3	7	9	0	1
4	5	7	10	0

只允许经过
1,2号顶点

	1	2	3	4
1	0	2	5	4
2	10	0	3	4
3	7	9	0	1
4	5	7	10	0

只允许经过
1,2,3号顶点

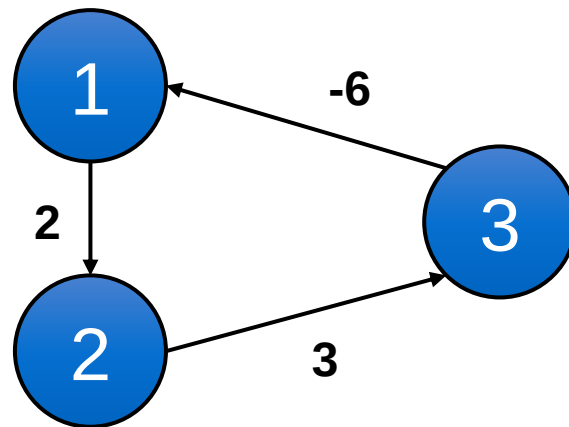
	1	2	3	4
1	0	2	5	4
2	9	0	3	4
3	6	8	0	1
4	5	7	10	0

允许经过
1,2,3,4号顶点



4.6.1 两种算法的区别

- ① Floyd算法是求任意两点之间的距离，是**多源**最短路，而Dijkstra算法是求一个顶点到其他所有顶点的最短路径，是**单源**最短路。
- ② Floyd算法属于动态规划，我们在写核心代码时候就是相当于推DP状态方程，Dijkstra算法属于贪心算法。
- ③ Dijkstra算法**时间复杂度**一般是 $O(n^2)$ ，Floyd算法时间复杂度是 $O(n^3)$ ，Dijkstra算法比Floyd算法快。
- ④ Floyd算法可以算带**负权**的，而Dijkstra算法是不可以算带负权的。但Floyd算法不能算**负权回路**。其实如果一个图中带有“负权回路”那么这个图则没有最短路。





4.7 最小生成树算法

概述：在一给定的无向图 $G = (V, E)$ 中， (u, v) 代表连接顶点 u 与顶点 v 的边（即），而 $w(u, v)$ 代表此边的权重，若存在 T 为 E 的子集且为无循环图，使得联通所有结点的 $w(T)$ 最小，则此 T 为 G 的**最小生成树**。

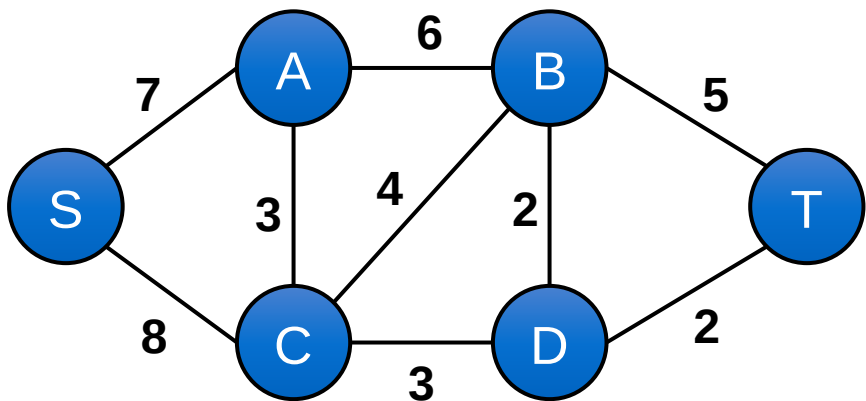
常用解决算法：Prim算法和Kruskal算法

克鲁斯卡尔（Kruskal）算法思想：

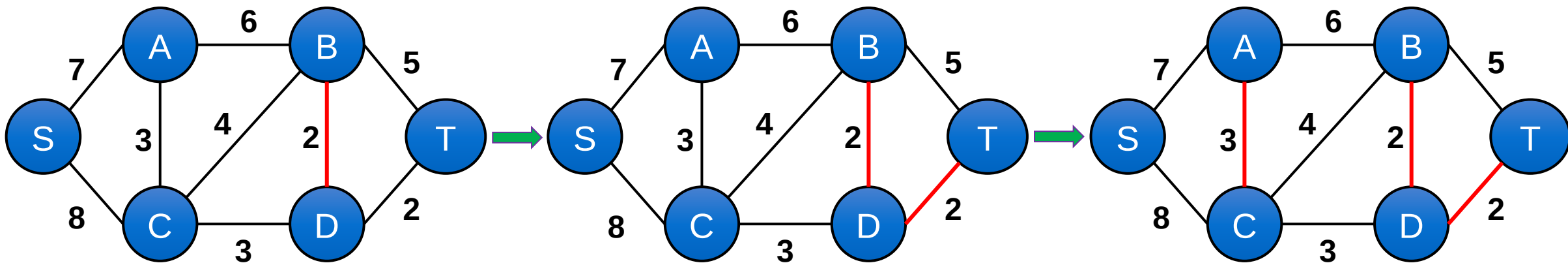
- ① 按照边的权值将边进行升序排序，然后从小到大一一判断
- ② 如果这个边与之前选择的所有边不会组成回路，就选择这条边，反之舍去
- ③ 不断判断，知道具有 n 个顶点的联通网筛选出来 $n-1$ 条边为止。



4.7 最小生成树案例（算法过程）

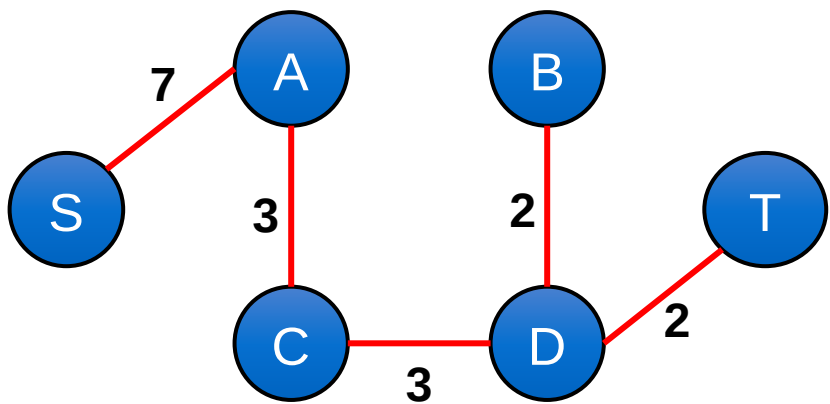
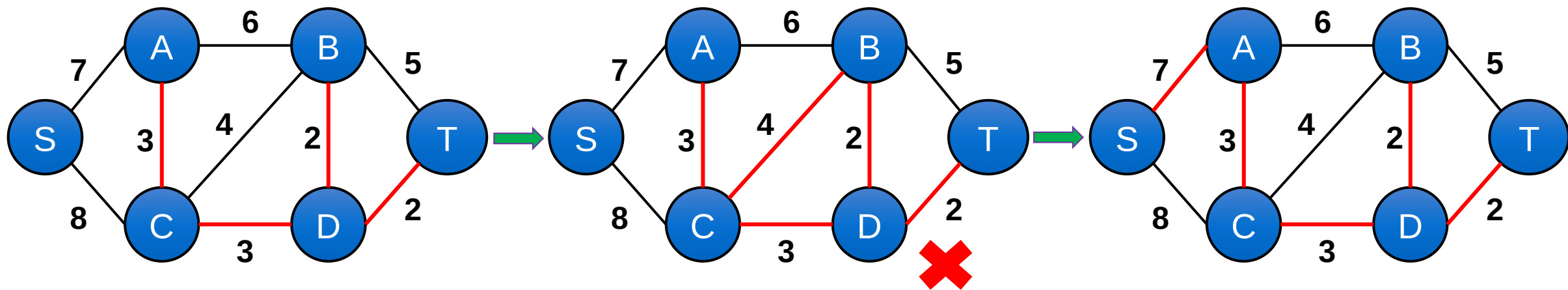


B,D	D,T	A,C	C,D	C,B	B,T	A,B	S,A	S,C
2	2	3	3	4	5	6	7	8





4.7 最小生成树案例（算法过程）



结果说明：如左下图所示，对于一个包含 6 个顶点的连通网，我们已经选择了 5 条边，这些边组成的生成树就是最小生成树。



4.8 动态规划(DP)法

在生产和科学实验中，有一类活动的过程，由于它的特殊性，可将过程分为若干个互相联系的阶段，在它的每一个阶段都需要作出决策，从而使整个过程达到最好的活动效果。**动态规划**是用来解决多阶段决策过程最优化的一种数量方法。其特点在于，可以把困难的多阶段决策问题变换成一系列互相联系较容易的单阶段问题，解决了这一系列较容易的单阶段问题，也就解决了多阶段决策问题。

基本思想与方程

不管在此最优策略上某个状态以前的状态和决策如何，对该状态来说，以后的所有决策必定构成最优子策略。即**最优策略的任意一个子策略也是最优的**。

逆序算法基本方程

$$\begin{cases} f_k(s_k) = \text{opt}\{r_k(s_k, u_k) + f_{k+1}(s_{k+1})\}, k = n, n-1, \dots, 1 \\ f_{n+1}(s_{n+1}) = 0 \end{cases}$$

顺序算法基本方程

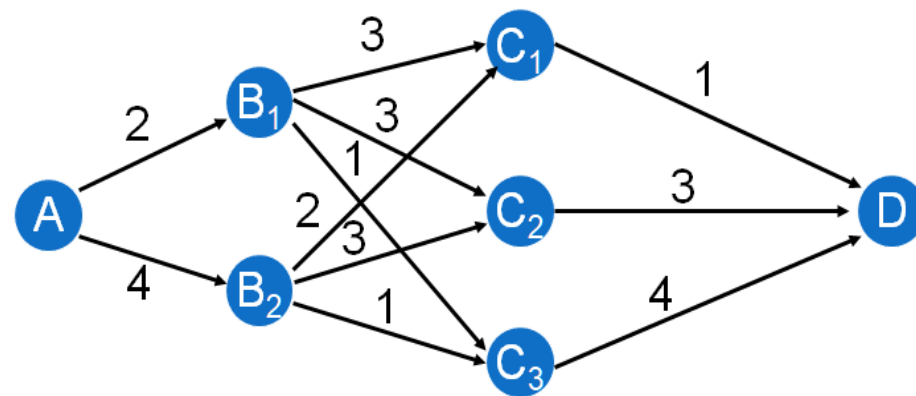
$$\begin{cases} f_k(s_k) = \text{opt}\{r_k(s_k, u_k) + f_{k-1}(s_{k-1})\}, k = 1, 2, \dots, n \\ f_0(s_0) = 0 \end{cases}$$



4.8 动态规划案例

动态规划
求解步骤

- 1、确定问题的阶段和编号
- 2、确定状态变量 s_k
- 3、确定决策变量 u_k
- 4、确定状态转移方程
- 5、确定指标函数



整个计算过程分三个阶段，从最后一个阶段开始。

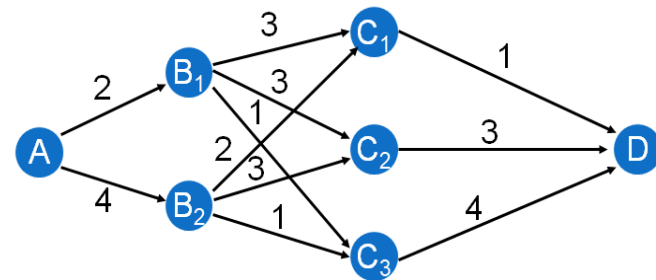
第三阶段 ($C \rightarrow D$): C有三条路线到终点D.

$$\text{显然有 } f_3(C_1) = 1 ; \quad f_3(C_2) = 3 ; \quad f_3(C_3) = 4$$



4.8 动态规划案例

第二阶段 (B→C) : B到C有六条路线。



$$f_2(B_1) = \min \begin{Bmatrix} r(B_1, C_1) + f_1(C_1) \\ r(B_1, C_2) + f_1(C_2) \\ r(B_1, C_3) + f_1(C_3) \end{Bmatrix} = \min \begin{Bmatrix} 3+1 \\ 3+3 \\ 1+4 \end{Bmatrix} = \begin{Bmatrix} 4 \\ 6 \\ 5 \end{Bmatrix} = 4$$

(最短路线为 $B_1 \rightarrow C_1 \rightarrow D$)

$$f_2(B_2) = \min \begin{Bmatrix} r(B_2, C_1) + f_1(C_1) \\ r(B_2, C_2) + f_1(C_2) \\ r(B_2, C_3) + f_1(C_3) \end{Bmatrix} = \min \begin{Bmatrix} 2+1 \\ 3+3 \\ 1+4 \end{Bmatrix} = \begin{Bmatrix} 3 \\ 6 \\ 5 \end{Bmatrix} = 3$$

(最短路线为 $B_2 \rightarrow C_1 \rightarrow D$)

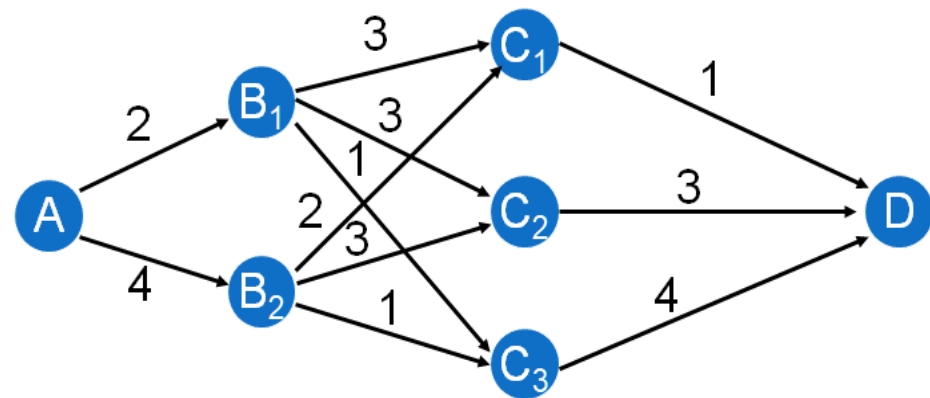


4.8 动态规划案例

第一阶段 ($A \rightarrow B$) : A到B有二条路线。

$$f_1(A) = \min \left\{ \begin{array}{l} r(A, B_1) + f_2(B_1) \\ r(A, B_2) + f_2(B_2) \end{array} \right\} = \min \left\{ \begin{array}{l} 2+4 \\ 4+3 \end{array} \right\} = 6$$

最短路线: $A \rightarrow B_1 \rightarrow C_1 \rightarrow D$ 路长: 6



1. **阶段**: 原问题分成若干相互联系的子问题, 每个子问题称为一个阶段。如本例是可分为3个阶段来求解。
2. **状态**: 每阶段初可能的情形或位置, 它描述了原问题过程的状况。如本例中第二个阶段有二个状态。
3. **决策**: 当过程处于某一阶段的某个状态时, 可以作出不同的决定, 从而确定下一阶段的状态。如本例第二个阶段中, 若从状态B1出发, 可作出三种不同的决策, 其允许决策集合 $D_2(B_1) = \{C_1, C_2, C_3\}$. 若选择的点为C2, 则C2是状态B1在决策 $u_2(B_1)$ 作用下的一个新的状态, 记作 $u_2(B_1) = C_2$.
4. **策略**: 一个按顺序排列的决策组成的集合。从允许策略集合中找出达到最优效果的策略称为**最优策略**。
5. **状态转移方程**: 从 s_k 的某一状态值出发, 当决策变量 $u_k(s_k)$ 的取值决定后, 下一阶段状态变量 s_{k+1} 的取值也随之确定。从上一阶段的某状态值到下阶段某状态值的转移的规律称为状态转移率(方程)。记为 $s_{k+1} = T_k(s_k, u_k)$ 。
6. **指标函数和最优值函数**: 用来衡量所实现过程优劣的一种数量指标, 称为指标函数。指标函数的最优值, 称为最优值函数, 记作 $f_k(s_k)$ 。在不同的问题中, 指标函数的含义是不同的, 它可能是距离、利润、成本等。



4.9 无约束极值问题

标准模型 $\min f(X), \quad X \in E^n$

例如: $\min f(x_1, x_2) = \frac{3}{2}x_1^2 + \frac{1}{2}x_2^2 - x_1x_2 - 2x_1$

求解方法:

- (1) 梯度法 (最速下降法);
- (2) 牛顿法 (原始经典算法);
- (3) 共轭梯度法;
- (4) 变尺度法;
- (5) 直接法一步长加速法。

注: 一维搜索方法—斐波那契法, 0.618法。



4.9 无约束极值问题

迭代法基本思想：

给定一个初始估计 $X^{(0)}$ ，然后按某种规则（即算法）找出比 $X^{(0)}$ 更好的解 $X^{(1)}$ ，再按此规则找出比 $X^{(1)}$ 更好的解 $X^{(2)}$ ……。如此即可得到一个解的序列 $\{X^{(k)}\}$ 若这个解序列有极限 X^* ，即 $\lim_{k \rightarrow \infty} \|X^{(k)} - X^*\| = 0$ ，则称它收敛于 X^* 。

下降迭代法基本步骤：

- (1) 选定某一初始点 $X^{(0)}$ ，并定 $k := 0$ ；
- (2) 确定搜索方向 $P^{(k)}$ ；
- (3) 从 $X^{(k)}$ 出发，沿方向 $P^{(k)}$ 求步长 λ_k ，以产生下一个迭代点 $X^{(k+1)}$ ；
- (4) 检查得到的新点 $X^{(k+1)}$ 是否为极小点或近似极小点。若是，则停止迭代。否则令 $k := k + 1$ ，转回 (2) 继续进行迭代。

注1： 选取搜索方向 $P^{(k)}$ 是关键一步，有关各种算法的区分，主要在于确定搜索方向的方法不同。

注2： λ_k 一般采用一维搜索，微分法，线搜索等确定最佳步长。



4.9 无约束极值问题

最速下降法计算步骤：

- (1) 给定初始近似点 $X^{(0)}$ 及精度 $\varepsilon > 0$ ，若 $\|\nabla f(X^{(0)})\| \leq \varepsilon$ ，则 $X^{(0)}$ 即为所求；
- (2) 若 $\|\nabla f(X^{(0)})\| > \varepsilon$ ，求步长 λ_0 ，并计算 $X^{(1)} = X^{(0)} - \lambda_0 \nabla f(X^{(0)})$ ；
- (3) 一般地，设已迭代到点 $X^{(k)}$ ，若 $\|\nabla f(X^{(k)})\| \leq \varepsilon$ ，则 $X^{(k)}$ 即为所求的近似解；若 $\|\nabla f(X^{(k)})\| > \varepsilon$ ，求步长 λ_k ，并确定下一个近似点 $X^{(k+1)} = X^{(k)} - \lambda_k \nabla f(X^{(k)})$ 。
如此继续，直至达到要求的精度为止。

注：求最佳步长的方法之一

若 $f(X)$ 具有二阶连续偏导数，在 $X^{(k)}$ 作 $f(X^{(k)} - \lambda \nabla f(X^{(k)}))$ 的泰勒展开

$$f(X^{(k)} - \lambda \nabla f(X^{(k)})) \approx f(X^{(k)}) - \nabla f(X^{(k)})^T \lambda \nabla f(X^{(k)}) + \frac{1}{2} \lambda \nabla f(X^{(k)})^T H(X^{(k)}) \lambda \nabla f(X^{(k)})$$

对 λ 求导并令其等于零，则得近似最佳步长：

$$\lambda_k = \frac{\nabla f(X^{(k)})^T \nabla f(X^{(k)})}{\nabla f(X^{(k)})^T H(X^{(k)}) \nabla f(X^{(k)})}$$



4.10 约束极值问题

标准模型

$$\begin{aligned} \min \quad & f(X) \\ \begin{cases} h_i(X) = 0, & i = 1, 2, \dots, m \\ g_j(X) \geq 0, & j = 1, 2, \dots, l \end{cases} \end{aligned}$$

求解方法：

- (1) Kuhn-Tucker 条件（广义拉格朗日法）；
- (2) 可行方向法；
- (3) SUMT 外点法；
- (4) SUMT 内点法；
- (5) 计算机模拟等。



4.10 约束极值问题

外点法基本思想：

把目标函数 $f(X)$ 看成“价格”，约束条件看成某种“规定”，采购人可在规定范围内购置最便宜的东西，此外对违反规定制定了一种“罚款”政策，若符合规定，罚款为零；否则，要收罚款。此时，采购人付出的代价应是价格和罚款的总和。当罚款规定得很苛刻时，违反规定支付的罚款很高，这就迫使采购人符合规定。

外点法迭代步骤：

(1) 取 $M_1 > 0$ （例如取1），允许误差 $\varepsilon > 0$ ，并令 $k := 1$

(2) 求无约束极值问题的最优解： $\min_{X \in E^n} P(X, M_k) = P(X^{(k)}, M_k)$ 式中 $P(X, M_k) = f(X) + M_k \sum_{j=1}^l [\min(0, g_j(X))]^2$

(3) 若对某一个 $j(1 \leq j \leq l)$ 有 $-g_j(X^{(k)}) \geq \varepsilon$ ，则取 $M_{k+1} > M_k (M_{k+1} = 5M_k)$

令 $k := k + 1$ ，并转向第2步。否则，停止迭代，得 $X_{\min} \approx X^{(k)}$



4.10 约束极值问题

内点法基本思想：

内点法与外点法不同，它要求迭代过程始终在可行域内部进行。为此，我们把初始点取在可行域内部，并在可行域的边界上设置一道“障碍”，使迭代点靠近可行域的边界时，给出的新目标函数值迅速增大，从而使迭代点始终留在可行域内部。

内点法迭代步骤：

(1) 取 $r_1 > 0$ （例如取1），允许误差 $\varepsilon > 0$ 。

(2) 找出一可行内点 $X^{(0)} \in R_0$ ，并令 $k=1$ 。

(3) 构造障碍函数，障碍项可采用倒数函数（或对数函数）：
$$\bar{P}(X, r_k) = f(X) + r_k \sum_{j=1}^l \frac{1}{g_j(X)}$$

(4) 以 $X^{(k-1)} \in R_0$ 为初始点，对障碍函数进行无约束极小化（在 R_0 内）：
$$\min_{X \in R_0} \bar{P}(X, r_k) = \bar{P}(X^{(k)}, r_k),$$

(5) 检验是否满足收敛准则
$$r_k \sum_{j=1}^l \frac{1}{g_j(X^{(k)})} \leq \varepsilon$$
$$X^{(k)} = X(r_k) \in R_0$$

如果满足，则 $X^{(k)}$ 为原问题的近似极小解；否则，取 $r_{k+1} < r_k$ ($r_{k+1} = r_k / 5$)，

令 $k := k + 1$ ，转向第(3)步继续进行迭代。



4.11 小结

我们碰到一个新问题时，如何去构造它的算法呢？肯定想发现一个新的有效的算法，对所有的输入条件总能得到一个最优解。如果我们怀疑这个问题是NP完全的，那怎么办呢？如果一下找不到有效的算法，可以考虑先把问题限制在某些**特殊情况或极端情况**，在研究了特殊情况的结果后，再逐步**推广到一般**的结果。我们习惯地把当前的问题归结到一个已知的问题，但这样在处理过程中可能会丢失某些结构。

如果碰到的问题输入规模很大，又很复杂，设法把这些输入分成几个子集合，从而得到几个可以分别求解的子问题，在分别地解决各个子问题后，还可以找到适当的方法把它们合并成原问题的解。这是分而治之的策略。

我们也可以考虑用一些通用的方法来求解这个问题，如分支定界法、动态规划或割平面法等。



4.12 优化类常用经典算法总结

问题或模型	常用算法（不限）	说明	软件工具（不限）
线性规划	单纯形法	最优解	Lingo、Matlab、Python
线性整数规划	分支定界法、割平面法	最优解	Lingo、Matlab、Python
非线性整数规划	蒙特卡洛法	满意解	Matlab、Lingo、Python
0-1整数规划	隐枚举法	不是穷举法；最优解	Lingo、Matlab、Python
非线性规划	梯度法、变尺度法、罚函数法	无固定解法，通常只能求满意解；常用现代优化算法	Matlab、Python
动态规划	逆序解法、贪婪算法	状态转移方程是关键	Matlab、Python
最短路问题	迪克斯特拉（Dijkstra）算法、Floyd算法	可以建立0-1规划模型	Matlab 、Lingo、Python
TSP问题	改良圈算法、隐枚举法	可以建立0-1规划模型，注意破圈	Matlab 、Lingo、Python
图论模型（最小生成树、网络最大流等）	避圈法、破圈法、标号法等	建立规划模型或图论模型	Matlab 、Lingo、Python
排队论(OR)	计算机模拟	确定数据的分布规律再模拟	Matlab、Python
博弈论(OR)	合作与非合作博弈、静态与动态博弈	Nash均衡是非常重要的思想	Matlab、Python



5. 启发式算法 (7.16介绍)

最优算法有时是有效的，也有些却十分麻烦，计算量非常大，以至于实际上是不可行的。实际求解问题要考虑经济效果，最优难以达到，次最优已经足够好的了。这就考虑采用**启发式算法（近似算法）**。分析人员必须运用自己的感知和洞察力，从与其有关而较基本的模型及算法中寻求其间的联系，从中得到启发，去发现适于解决该问题的思路和途径，这种方法称为启发式方法，由此建立的算法称为启发式算法。

比较著名的启发式算法有遗传算法、模拟退火算法、神经网络算法、粒子群算法、蚁群算法、禁忌搜索算法、拉格朗日松弛算法、混合优化算法等。



6. 计算机模拟

计算机模拟（仿真），是根据已知的信息和知识（如数学、物理规律），利用计算机模拟现实情况或系统的演变过程，发现新的知识或规律，从而解决问题的一种方法。独立于理论研究与实验研究的认识世界的第三种方法。

计算机仿真的特点：代价小、时间短、可重复、参数设置灵活。

计算机仿真的优点：计算机模拟可以反复进行，改变系统的结构和系数都比较容易。在实际问题中，面对一些带随机因素的复杂系统，用分析方法建模常常需要作许多简化假设，与面临的实际问题可能相差甚远，以致解答根本无法应用。这时，计算机模拟几乎成为唯一的选择。



6. 计算机模拟分类

什么问题适合于使用计算机仿真求解？

概括起来，适合于使用计算机仿真求解的问题通常具备以下两个特征之一：

- ★ 局部规律已知，全局表现未知。
- ★ 具有随机因素。

根据以上两大特征把计算机仿真问题分为四大类：

- A. 物理规律仿真： 只具备特征一，并且与时间无关（或关系不大），如喷泉问题、电路问题等。
- B. 系统演变仿真： 只具备特征一，且与时间有关，如洪水漫延、生命游戏等。
- C. 蒙特卡罗方法： 只具备特征二，如高尔顿板钉实验、复杂图形面积计算等。
- D. 离散事件仿真： 同时具备两个特征，如电梯调度、交通流模拟、飞机登记顺序等。



6. 计算机模拟示例-生命游戏

生命游戏 (Life Game) 是由数学家康维于1970年发明的。是一类特殊的细胞自动机。在简单的规则下，不同初始状态将演变出五彩缤纷的画面。

游戏背景

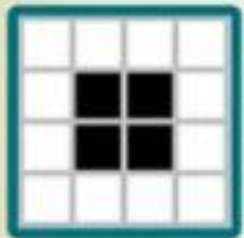
- ★ 生命游戏在一个方形细胞网格中进行，类似于棋盘。
- ★ 细胞分为“活细胞”和“死细胞”。
- ★ 细胞的“邻居”是指它周围的8个方格中的细胞。

游戏规则

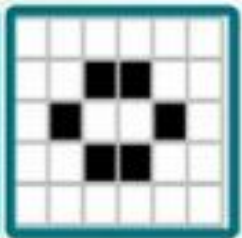
- ★ 一个死细胞恰有3个活邻居的话，就成为一个活细胞(诞生)。
- ★ 一个活细胞有2个或3个活邻居的话，就继续活下去(生存)。
- ★ 其他情况下，细胞死去或是保持死的状态(孤独或拥挤)。



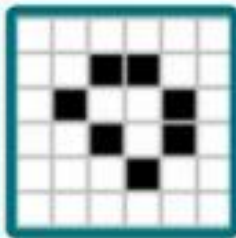
6. 计算机模拟示例-生命游戏



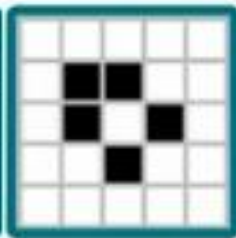
砖块



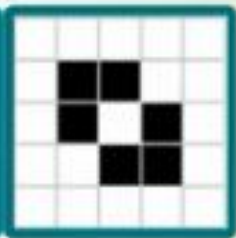
蜂窝



面包

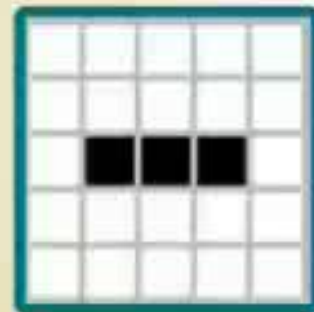


小船

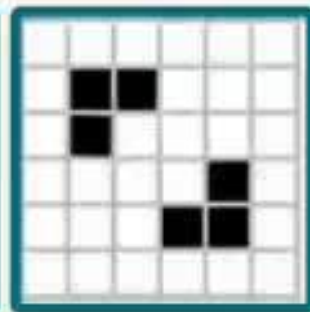


大船

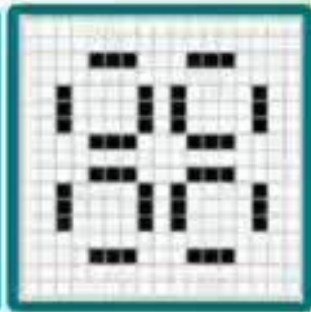
生者恒生 (Stilllives)



眨眼者

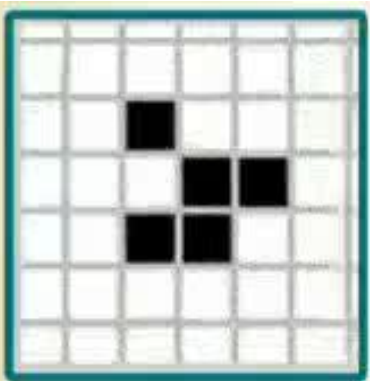


灯塔

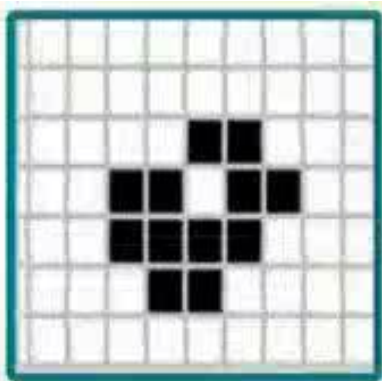


脉冲星

震荡 (Oscillators)

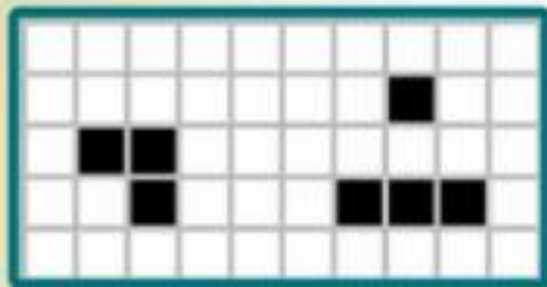


滑翔机

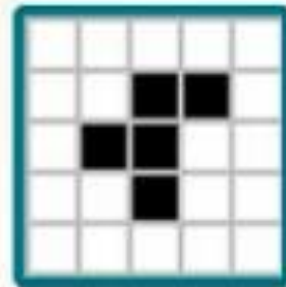


轻型飞船

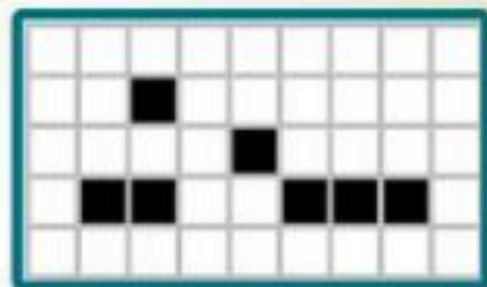
飞船 (Spaceships)



顽固者



五格拼板



橡树果

寿星 (Methuselahs)



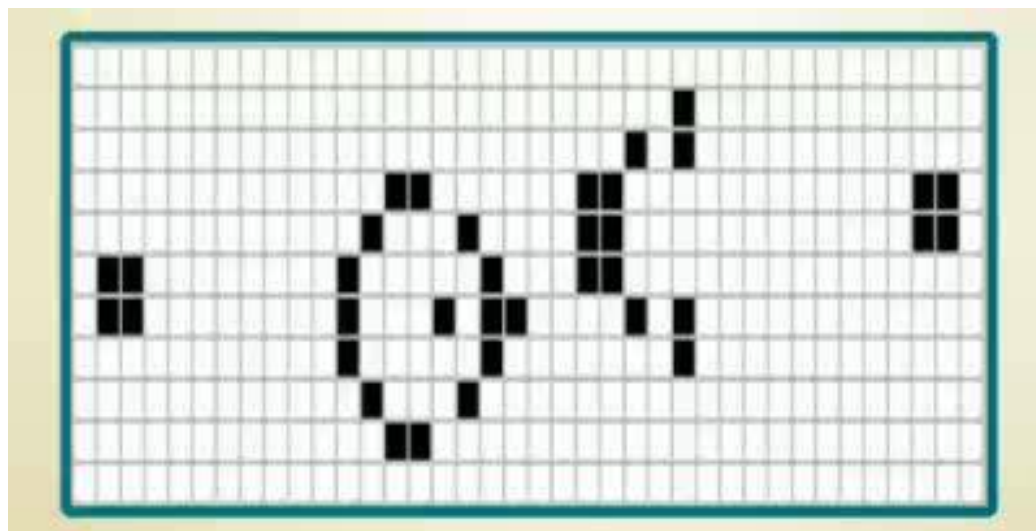
6. 计算机模拟示例-生命游戏

- ◆ 顽固者经过130代后消失。
- ◆ 五格拼板需要1103代才能稳定下来。
- ◆ 橡树果需要5206代才能稳定下来，生成633个活细胞，其中包括13个飞船。



康维猜想：任何游戏都不能无限增长下去？

他在《科学美国人》数学游戏专栏里面悬赏。



著名的反例：滑翔机发射器



6. 计算机模拟示例-火炮打击

在我方某前沿防守地域，敌人以一个炮排（含两门火炮）为单位对我方进行干扰和破坏。为躲避我方打击，敌方对其阵地进行了伪装并经常变换射击地点。经过长期观察发现，我方指挥所对敌方目标的指示有50%是准确的，而我方火力单位，在指示正确时，有 $1/3$ 的射击效果能毁伤敌人一门火炮，有 $1/6$ 的射击效果能全部消灭敌人。

现在希望能用某种方式把我方将要对敌人实施的20次打击结果显现出来，确定有效射击的比率及毁伤敌方火炮的平均值。

分析：这是一个概率问题，可以通过理论计算得到相应的概率和期望值。但这样只能给出作战行动的最终静态结果，而显示不出作战行动的动态过程。



6. 计算机模拟示例-火炮打击

1. 问题分析 需要模拟以下两件事

[1] 观察所对目标的指示正确与否模拟试验有两种结果，每一种结果出现的概率都是 $1/2$ 。**可用投掷一枚硬币的方式予以确定**，当硬币出现正面时为指示正确，反之为不正确。

[2] 当指示正确时，我方火力单位的射击结果情况模拟试验有三种结果：毁伤一门火炮的可能性为 $1/3$ ，毁伤两门的可能性为 $1/6$ ，没能毁伤敌火炮的可能性为 $1/2$ 。

这时**可用投掷骰子的方法来确定**：

如果出现的是 1、2、3 三个点：则认为没能击中敌人；

如果出现的是 4、5 点：则认为毁伤敌人一门火炮；

若出现的是 6 点：则认为毁伤敌人两门火炮。

2. 符号假设

i ：要模拟的打击次数； k_1 ：没击中敌人火炮的射击总数；

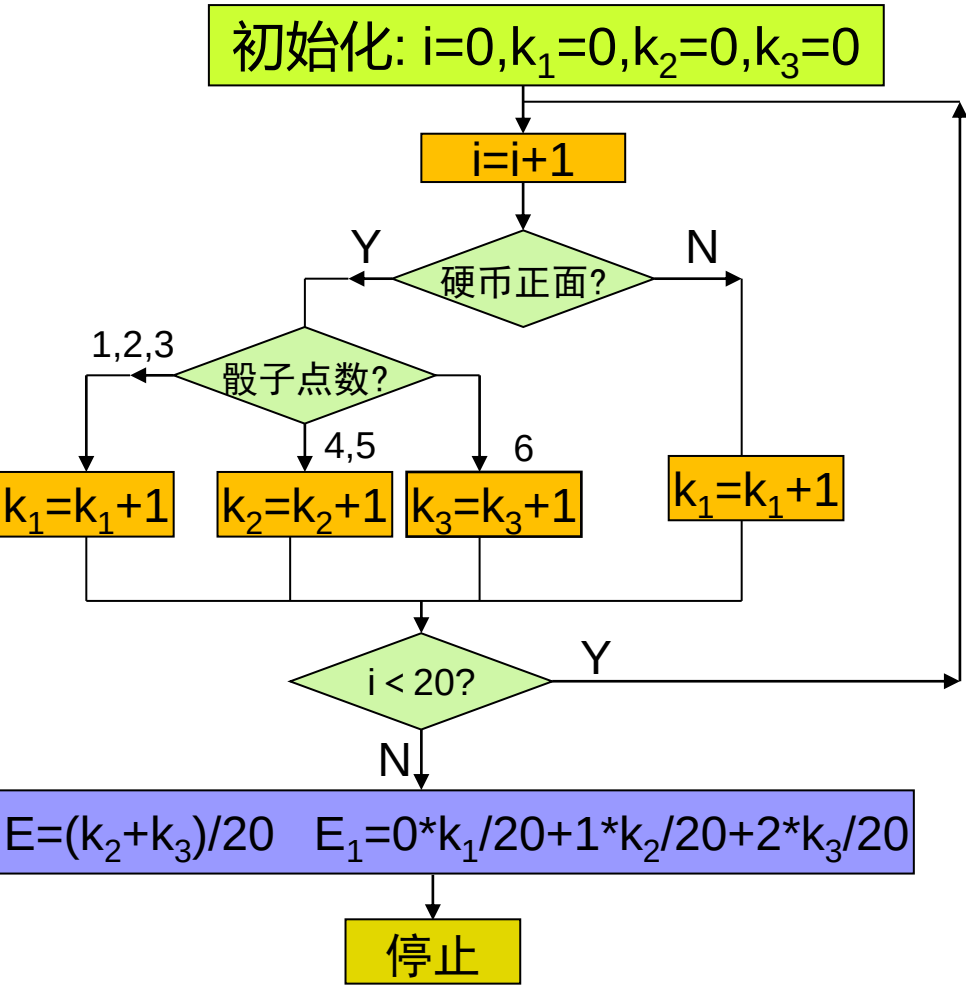
k_2 ：击中敌人一门火炮的射击总数； k_3 ：击中敌人两门火炮的射击总数；

E ：有效射击比率； E_1 ：20次射击平均每次毁伤敌人的火炮数。



6. 计算机模拟示例-火炮打击

3. 模拟框图



4. 模拟结果与理论结果

试验序号	投硬币结果	指示正确	指示不正确	掷骰子结果	消灭敌人火炮数		
					0	1	2
1	正	√		4		√	
2	正	√		4		√	
3	反		√		√		
4	正	√		1	√		
5	正	√		2	√		
6	反		√		√		
7	正	√		3	√		
8	正	√		6			√
9	反		√		√		
10	反		√		√		

试验序号	投硬币结果	指示正确	指示不正确	掷骰子结果	消灭敌人火炮数		
					0	1	2
11	正	√		2	√		
12	反		√		√		
13	正	√		3	√		
14	反		√		√		
15	正	√		6			√
16	正	√		4		√	
17	正	√		2	√		
18	正	√		4		√	
19	反		√		√		
20	正	√		6			√

分类项目	无效射击	有效射击	平均值
模拟	0.65	0.35	0.5
理论	0.75	0.25	0.33

虽然模拟结果与理论计算不完全一致，但它却能更加真实地表达实际战斗动态过程。



6. 计算机模拟示例-火炮打击

5. 理论计算

$$j = \begin{cases} 0 & \text{观察所对目标指示不正确} \\ 1 & \text{观察所对目标指示正确} \end{cases}$$

设： A_0 ：射中敌方火炮的事件；

A_1 ：射中敌方一门火炮的事件；

A_2 ：射中敌方两门火炮的事件。

则由全概率公式：

$$\begin{aligned} E &= P(A_0) = P(j=0)P(A_0 | j=0) + P(j=1)P(A_0 | j=1) \\ &= 0.5 \times 0 + 0.5 \times 0.5 = 0.25 \end{aligned}$$

$$P(A_1) = P(j=0)P(A_1 | j=0) + P(j=1)P(A_1 | j=1) = 0.1667$$

$$P(A_2) = P(j=0)P(A_2 | j=0) + P(j=1)P(A_2 | j=1) = 0.0833$$

$$E_1 = 1 \times 0.1667 + 2 \times 0.0833 \approx 0.33$$

说明：用蒙特卡洛方法进行计算机模拟的步骤：

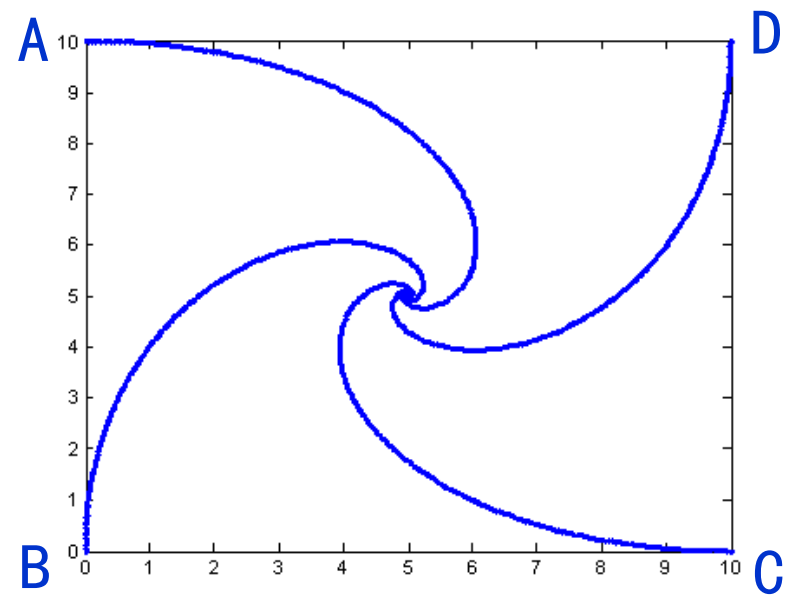
- [1] 设计一个逻辑框图，即模拟模型。这个框图要正确反映系统各部分运行时的逻辑关系。
- [2] 模拟随机现象。可通过具有各种概率分布的模拟随机数来模拟随机现象。



6. 计算机模拟示例-追逐问题

连续系统模拟：状态随时间连续变化的系统称为连续系统。对连续系统的计算机模拟只能是近似的，只要这种近似达到一定的精度，也就可以满足要求。

追逐问题：如图，正方形ABCD的四个顶点各有一人。在某一时刻，四人同时出发以匀速 $v = 1$ 米/秒按顺时针方向追逐下一人，如果他们始终保持对准目标，则最终按螺旋状曲线于中心点O。试求出这种情况下每个人的行进轨迹。





6. 计算机模拟示例-追逐问题

求解过程:

1. 建立平面直角坐标系: $A(x_1, y_1)$, $B(x_2, y_2)$, $C(x_3, y_3)$, $D(x_4, y_4)$.

2. 取时间间隔为 Δt , 计算每一点在各个时刻的坐标。

设某点在 t 时刻的坐标为: (x_i, y_i)

则在时刻 $t + \Delta t$ 的坐标为: $(x_i + v\Delta t \cos \alpha, y_i + v\Delta t \sin \alpha)$

其中

$$\cos \alpha = \frac{x_{i+1} - x_i}{d}, \quad \sin \alpha = \frac{y_{i+1} - y_i}{d}$$

$$d = \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}$$

3. 取足够小的 ε , $d < \varepsilon$ 时结束算法。

4. 对每一个点, 连接它在各时刻的位置, 即得所求运动轨迹。

计算程序:

```
v=1;
dt=0.05;
x=[0 0 10 10];
y=[0 10 10 0];
for i=1:4
    plot(x(i),y(i),'r'),hold on
end
d=20;
while(d>0.1)
    x(5)=x(1);y(5)=y(1);
    for i=1:4
        d=sqrt((x(i+1)-x(i))^2+(y(i+1)-y(i))^2);
        x(i)=x(i)+v*dt*(x(i+1)-x(i))/d;
        y(i)=y(i)+v*dt*(y(i+1)-y(i))/d;
        plot(x(i),y(i),'r'),hold on
    end
end
end
```



6. 计算机模拟示例-排队问题

排队论主要研究随机服务系统的工作过程。

在排队系统中，服务对象的到达时间和服务时间都是随机的。排队论通过对每个个别的随机服务现象的统计研究，找出反映这些随机现象平均特性的规律，从而为设计新的服务系统和改进现有服务系统的工作提供依据。

对于排队服务系统，顾客常常注意排队的人是否太多，等候的时间是否长，而服务员则关心他空闲的时间是否太短。于是人们常用排队的长度、等待的时间及服务利用率等指标来衡量系统的性能。



6. 计算机模拟示例-排队问题

单服务员的排队模型：在某商店有一个售货员，顾客陆续来到，售货员逐个地接待顾客。当到来的顾客较多时，须排队等待，被接待后的顾客便离开商店。设：

1. 顾客到来间隔时间服从参数为0.1的指数分布.
2. 对顾客的服务时间服从 $[4, 15]$ 上的均匀分布.
3. 排队按先到先服务规则，队长无限制.

假定一个工作日为8小时，时间以分钟为单位。

[1]模拟一个工作日内完成服务的个数及顾客平均等待时间 t .

[2]模拟100个工作日，求出平均每日完成服务的个数及每日顾客的平均等待时间。



6. 计算机模拟示例-排队问题

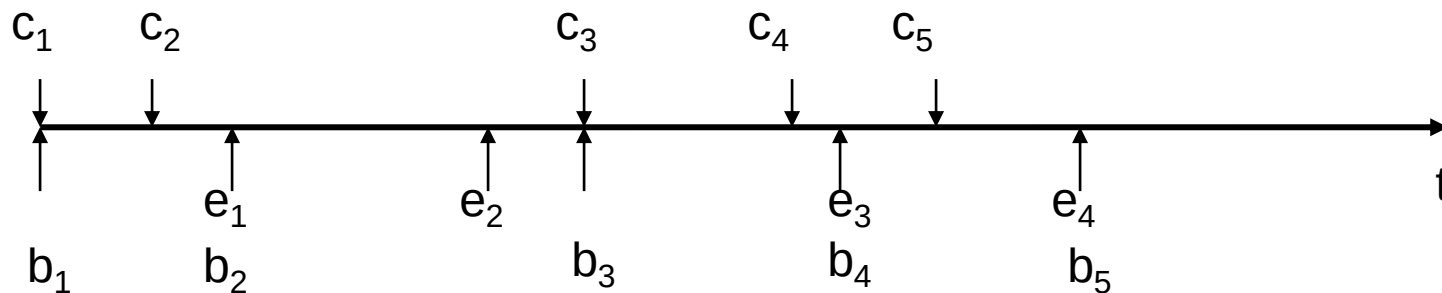
[1] 系统的假设:

- (1) 顾客源是无穷的;
- (2) 排队的长度没有限制;
- (3) 到达系统的顾客按先后顺序依次进入服务.

[2] 符号说明

c_i : 第 i 个顾客的到达时刻; b_i : 第 i 个顾客开始服务时刻; e_i : 第 i 个顾客服务结束时刻;

w : 总等待时间; x_i : 第 $i-1$ 个顾客与第 i 个顾客之间到达的间隔时间; y_i : 对第 i 个顾客的服务时间

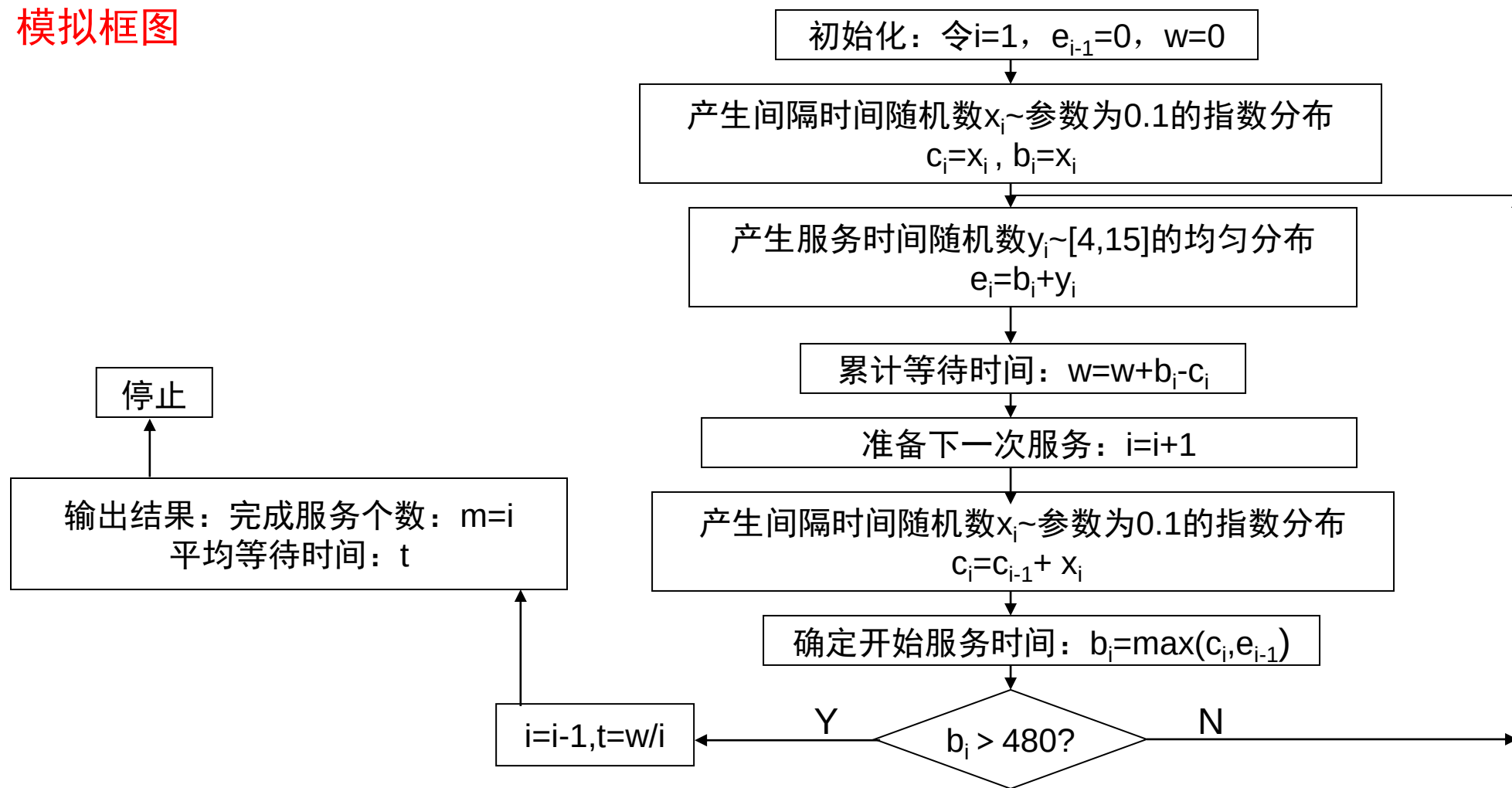


相关公式: $c_i = c_{i-1} + x_i$, $e_i = b_i + y_i$, $b_i = \max(c_i, e_{i-1})$



6. 计算机模拟示例-排队问题

[3] 模拟框图





6. 计算机模拟示例-非线性规划问题

对于非线性规划问题：

$$\begin{aligned} \min \quad & f(X) \quad X \in E^n \\ \text{s.t.} \quad & \begin{cases} g_i(X) \geq 0, i = 1, 2, \dots, m \\ a_i \leq x_i \leq b_i, i = 1, 2, \dots, n \end{cases} \end{aligned}$$

用蒙特卡洛法求解的**基本思想**是：在估计的区域

$$\{(x_1, x_2, \dots, x_n) \mid x_i \in [a_i, b_i], i = 1, 2, \dots, n\}$$

内随机取若干试验点，然后从试验点中找出可行点，再从可行点中选择最小点。

基本假设 试验点的第 j 个分量 x_j 服从 $[a_j, b_j]$ 内的均匀分布。

符号假设

- P: 试验点总数; MAXP: 最大试验点总数;
- K: 可行点总数; MAXK: 最大可行点数;
- X*: 迭代产生的最优点;
- Q: 迭代产生的最小值 $f(X^*)$ ，其初始值为计算机所能表示的最大数。



6. 计算机模拟示例-非线性规划问题

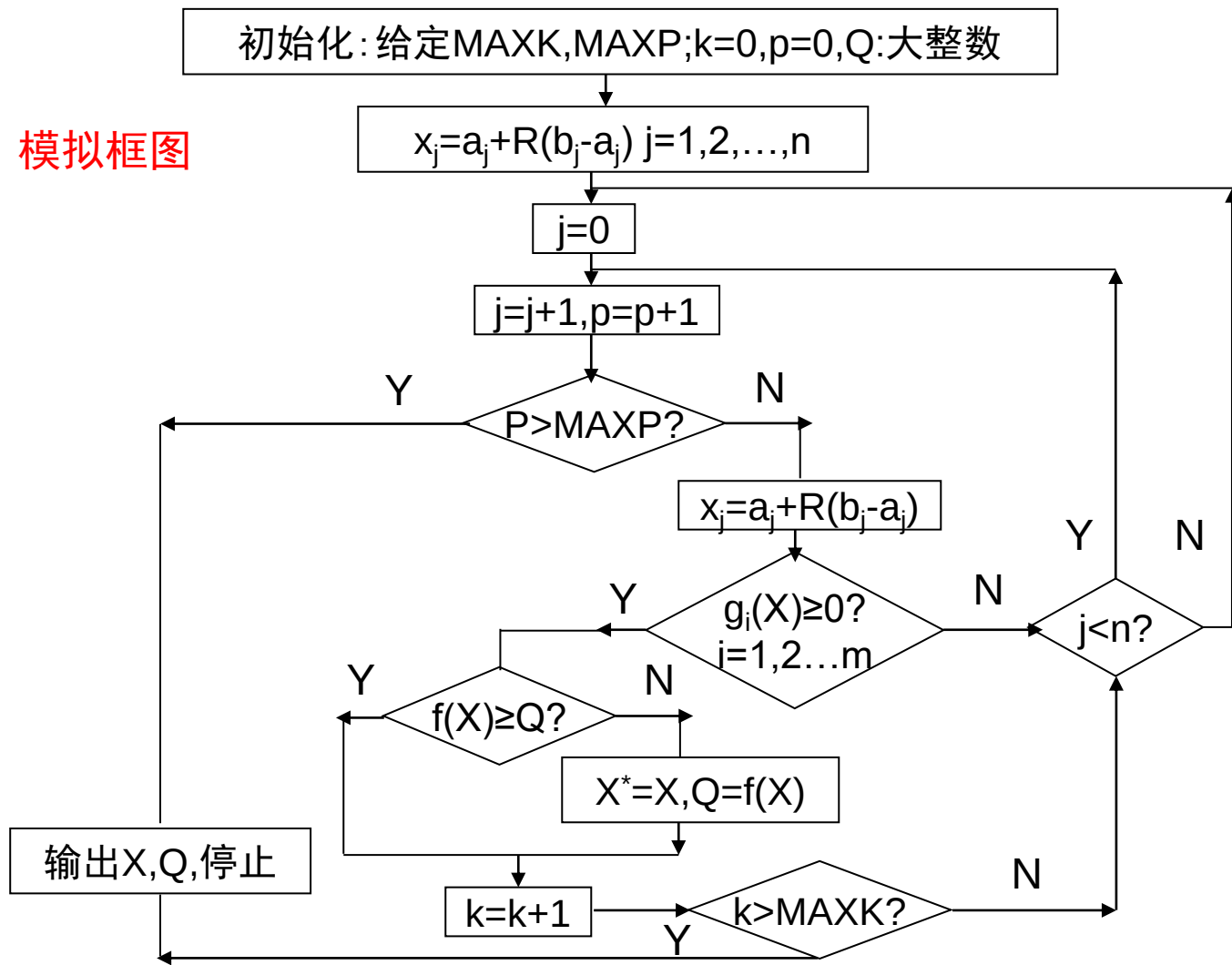
求解过程:

先产生一个随机数作为初始试验点,以后则将上一个试验点的第 j 个分量随机产生,其它分量不变而产生一新的试验点.这样,每产生一个新试验点只需一个新的随机数分量.

当 $K > \text{MAXK}$ 或 $P > \text{MAXP}$ 时停止迭代.

注: R 为0到1的随机数。

模拟框图





6. 计算机模拟示例-非线性规划问题

$$\begin{aligned} \max z &= -2x_1^2 - x_2^2 + x_1x_2 + 8x_1 + 3x_2 \\ \text{s.t.} \quad &\begin{cases} 3x_1 + x_2 = 10 \\ x_1 > 0 \\ x_2 > 0 \end{cases} \end{aligned}$$

求解此问题在Matlab中编程，共需三个M文件：
`randlp.m`, `mylp.m`, `lpconst.m`. 主程序为`randlp.m`.

```
% mylp.m
function z=mylp(x)                %目标函数
z=2*x(1)^2+x(2)^2-x(1)*x(2)-8*x(1)-3*x(2); %转化为求最小值问题
% lpconst.m
function lpc=lpconst(x)           %约束条件
if 3*x(1)+x(2)-10<=0.5 & 3*x(1)+x(2)-10>=-0.5 %约束条件的误差为
    lpc=1;
else
    lpc=0;
end
```

```
% randlp.m
function [sol,r1,r2]=randlp(a,b,n) % 随机模拟解非线性规划
debug=1;
a=0;                                % 试验点下界
b=10;                               % 试验点上界
n=1000;                             % 试验点个数
r1=unifrnd(a,b,n,1);                % n×1阶的 [a,b] 均匀随机数矩阵
r2=unifrnd(a,b,n,1);
sol=[r1(1) r2(1)];
z0=inf;
for i=1:n
    x1=r1(i);
    x2=r2(i);
    lpc=lpconst([x1 x2]);
    if lpc==1
        z=mylp([x1 x2]);
        if z<z0
            z0=z;
            sol=[x1 x2];
        end
    end
end
```



数学建模暑期培训

Thank you!



西南交通大学
Southwest Jiaotong University

中国 · 成都